COMSATS University Islamabad, Sahiwal Campus

CITY SIZZLE FOOD APP

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Amna Khadim	**CIIT/FA22-BCS-206/SWL**
Ahmad Nawaz	**CIIT/FA22-BCS-129/SWL**

Supervisor

Sir Umar Khan

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this **City Sizzle** project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Amna Khadim ----------

Ahmad Nawaz ----------

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) "**City Sizzle Food App**" was developed by **Amna Khadim (CIIT/FA22-BCS-206)**, and **Ahmad Nawaz (CIIT/FA22-BCS-129)** under the supervision of "**Sir Umar Khan**" that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Sir Umar Khan

(Lecturer)

External Examiner

Dr. Ansar Munir Shah

(Assistant Professor)

Department Of Computer Science

USP Multan

Head of Department

Dr. Zafar Iqbal Roy

(Department of Computer Science)

Executive Summary

City Sizzle is a modern mobile-based food ordering application designed to provide users with a seamless and efficient food ordering experience. The system enables customers to browse menus, customize food items, place orders, and track deliveries in real-time. It also integrates secure digital payment systems including JazzCash and EasyPaisa, ensuring fast and reliable transactions.

The application is developed using Flutter for the frontend and Supabase as the backend service, providing scalable database management and real-time communication. The system includes an advanced admin panel that allows administrators to manage menu items, process orders, and control system operations efficiently.

City Sizzle addresses common issues in existing food delivery systems such as delayed orders, lack of transparency, and poor user experience. It introduces improved features like real-time tracking, accurate menu updates, and secure payment processing.

The project follows an Agile development methodology, allowing incremental development and continuous improvement. Various software engineering concepts such as Object-Oriented Design, UML diagrams, and database modeling have been applied to ensure a robust and scalable system.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task of **City Sizzle**.

We are greatly indebted to our project supervisor “Sir Umar Khan”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful.

We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Amna Khadim ----------

Ahmad Nawaz ----------

Abbreviations

Abbreviation	Full Form	Description
API	Application Programming Interface	A set of rules that allows communication between frontend and backend systems
UI	User Interface	The visual layout through which users interact with the application
UX	User Experience	Overall experience of the user while using the system
SRS	Software Requirements Specification	Document describing system requirements and functionalities
SDD	Software Design Document	Document describing system design and architecture
DB	Database	Structured storage of system data
SQL	Structured Query Language	Language used to manage and query relational databases
CRUD	Create, Read, Update, Delete	Basic operations performed on data
MVC	Model View Controller	Architectural pattern used in system design

DFD	Data Flow Diagram	Diagram showing flow of data within the system

UML	Unified Modeling Language	Standard language for system modeling and diagrams
OTP	One Time Password	Security code used for authentication
SDK	Software Development Kit	Tools used for developing applications
IDE	Integrated Development Environment	Software used for coding and development
REST	Representational State Transfer	API architecture used for communication
JSON	JavaScript Object Notation	Data format used for data exchange
HTTP	HyperText Transfer Protocol	Protocol used for web communication
HTTPS	HyperText Transfer Protocol Secure	Secure version of HTTP
GPS	Global Positioning System	Used for location tracking (future feature)
QA	Quality Assurance	Process of ensuring system quality
UAT	User Acceptance Testing	Final testing phase by end users
KPI	Key Performance Indicator	Metrics used to evaluate system performance

CI/CD	Continuous Integration / Continuous Deployment	Process for automated development and deployment
SaaS	Software as a Service	Cloud-based software delivery model
DBMS	Database Management System	Software used to manage databases (e.g., Supabase)
POS	Point of Sale	System used for processing payments
OTP	One Time Password	Used for secure login/verification
COD	Cash on Delivery	Payment method where payment is made upon delivery
FYP	Final Year Project	Academic project for degree completion

Table of Contents

List Of Figures

List Of Tables

Chapter 1

Introduction

1 Introduction

In recent years, the rapid advancement of digital technologies and widespread adoption of smartphones have significantly transformed the way people interact with services in their daily lives. One of the most prominent areas affected by this transformation is the food industry, particularly the food delivery sector. Traditional methods of ordering food, such as phone calls or physical visits to restaurants, have gradually been replaced by mobile applications that allow users to browse menus, place orders, and make payments from the comfort of their homes.

City Sizzle is a modern mobile-based food ordering application designed to provide a seamless, efficient, and user-friendly experience to customers. The application is developed using the Flutter **[1]** framework for frontend development and Supabase **[2]** as the backend service provider, ensuring real-time data management and scalability.

The primary goal of this project is to create a centralized platform where customers can explore a wide range of food options, customize their orders, and complete transactions using secure digital payment methods such as JazzCash and EasyPaisa. In addition to customer functionalities, the system also includes a comprehensive admin panel that allows administrators to manage menu items, process orders, and monitor system activities effectively.

This project not only focuses on delivering a functional application but also emphasizes the implementation of modern software engineering principles, including modular design, scalability, and security. The development process follows an Agile methodology, allowing continuous improvements and iterative development throughout the project lifecycle.

1.1 Background of the Study

The evolution of online food ordering systems can be traced back to the early 2000s when restaurants began offering basic online ordering through websites. However, with the introduction of smartphones and mobile applications, the demand for more sophisticated and user-friendly systems increased significantly.

Today, platforms such as food delivery applications have become an integral part of urban lifestyles. These platforms provide convenience, time-saving benefits, and access to a wide variety of cuisines. Despite their popularity, many existing systems suffer from limitations

such as delayed order processing, lack of transparency, inaccurate menu information, and inefficient communication between users and restaurants. In developing regions, including Pakistan, the adoption of digital payment systems like JazzCash and EasyPaisa [3] has further accelerated the growth of online services. However, many food ordering systems still fail to fully integrate these payment solutions effectively, leading to inconvenience for users.

City Sizzle is developed as a response to these challenges. It aims to provide a more efficient, transparent, and reliable system that addresses the shortcomings of existing platforms while leveraging modern technologies to enhance user experience.

1.2 Relevance to Course Modules

The development of the City Sizzle application is closely aligned with multiple core courses studied during the Bachelor of Science in Computer Science program. This project serves as a practical implementation of theoretical concepts learned throughout the degree and demonstrates the integration of knowledge from various domains of computer science

1.2.1 Software Engineering

The principles of software engineering play a central role in the development of City Sizzle. The project follows a structured approach that includes requirement gathering, system design, implementation, testing, and maintenance.

Key concepts applied from this course include:

- Preparation of Software Requirements Specification (SRS)
- Development of Software Design Document (SDD)
- Use of software development lifecycle models (Agile methodology)
- Implementation of modular and maintainable code structure
- Risk analysis and project planning

The use of SRS and SDD documents ensures that the system requirements are clearly defined and properly translated into a working design, reducing ambiguity and improving development efficiency.

1.2.2 Database Management Systems

Database management [4] is a fundamental component of the City Sizzle system, as it handles all user, order, and transaction data.

Concepts applied from this course include:

- Design of relational database schemas
- Use of structured query language (SQL)
- Data normalization to reduce redundancy
- Efficient data storage and retrieval
- Maintaining data integrity and consistency

The system utilizes Supabase, which is based on PostgreSQL, to manage data effectively. Tables such as users, orders, food items, and payments are structured to ensure efficient system performance.

1.2.3 Mobile Application Development

City Sizzle is developed as a mobile application using Flutter, which directly relates to the concepts learned in mobile app development courses.

Key concepts include:

- Cross-platform application development
- User interface (UI) design and layout
- Navigation and screen management
- Integration with backend services
- Handling user inputs and events

Flutter enables the development of a responsive and visually appealing application, ensuring a smooth user experience.

1.2.4 Human Computer Interaction (HCI)

The design of the City Sizzle application is influenced by Human Computer Interaction principles, which focus on improving user experience and usability.

Concepts applied include:

- Designing intuitive and user-friendly interfaces
- Ensuring consistency in layout and navigation
- Minimizing user effort in completing tasks
- Providing clear feedback and system responses
- Enhancing accessibility and usability

These principles ensure that users can easily navigate the application and perform tasks such as ordering food and making payments without confusion.

1.2.5 Computer Networks

The system relies heavily on internet-based communication between the frontend and backend, making computer networking concepts highly relevant.

Key applications include:

- Client-server architecture
- API communication over HTTP/HTTPS
- Data transmission between mobile app and backend server
- Handling network requests and responses
- Ensuring secure communication

The interaction between the Flutter application and Supabase backend is managed through APIs, which require a strong understanding of networking principles.

1.2.6 Data Structures and Algorithms

Efficient handling of data within the system is achieved through the use of appropriate data structures and algorithms.

- Concepts applied include:
- Managing dynamic data such as cart items and orders
- Searching and filtering food items
- Optimizing performance for faster response times
- Implementing logical workflows for order processing

These concepts ensure that the application performs efficiently, even when handling multiple users and large amounts of data.

1.2.7 Cyber Security

Security is a critical aspect of the City Sizzle system, particularly due to the integration of online payment methods such as JazzCash and EasyPaisa.

Concepts applied include:

- Secure authentication and authorization
- Data encryption during transmission
- Protection against unauthorized access
- Secure handling of user credentials
- Ensuring safe payment transactions

These measures help in protecting user data and maintaining trust in the system.

1.3 Literature Review

The rapid advancement of mobile technologies and the increasing penetration of internet services have significantly transformed the food delivery industry, making online food ordering systems an essential part of modern lifestyles. Over the past decade, various digital platforms have emerged to facilitate convenient access to restaurant services, enabling users to browse menus, place orders, and complete transactions without physical interaction. Applications such as Foodpanda, Uber Eats, and DoorDash have played a significant role in shaping user expectations by introducing features such as real-time order tracking, digital payments, and user-friendly interfaces [1]. These systems have contributed to increased efficiency in service delivery and have established a new standard for convenience and accessibility in the food industry.

Despite their widespread adoption and continuous improvement, existing food delivery platforms still face several challenges that limit their overall effectiveness. One of the most commonly observed issues is the inconsistency in order processing, particularly during peak hours, which often results in delayed confirmations and late deliveries. Additionally, although many platforms offer multiple payment options, the integration of region-specific digital wallets is sometimes inadequate, especially in developing countries where services like JazzCash and EasyPaisa are widely used. Another concern relates to the user interface design of certain applications, where complex navigation structures and excessive information can

reduce usability, particularly for new users [2]. Furthermore, inaccuracies in menu availability and pricing information can lead to confusion and dissatisfaction among customers, indicating a lack of real-time synchronization between restaurant data and the application.

From a technical perspective, modern food delivery systems are built using a combination of frontend frameworks, backend services, and cloud-based databases to ensure scalability and performance. Technologies such as Flutter and React Native are commonly used for cross-platform mobile development, while backend solutions like Firebase and Supabase provide real-time database capabilities and API integration [6]. These technologies enable seamless communication between the client and server, allowing for efficient data exchange and system responsiveness. However, the effectiveness of these technologies largely depends on how well they are implemented within the system architecture, as poor integration can lead to performance bottlenecks and security vulnerabilities.

The analysis of existing systems reveals a gap between user expectations and system performance, particularly in terms of affordability, usability, and transparency. High service charges and delivery fees can discourage frequent usage, while the lack of clear communication regarding order status can reduce user trust in the system. Moreover, limited customization options and insufficient administrative controls can affect both user satisfaction and operational efficiency. These limitations highlight the need for a more optimized solution that prioritizes user experience, supports local payment systems, and ensures accurate and real-time data handling.In response to these challenges, the development of City Sizzle is aimed at providing an improved food ordering platform that addresses the shortcomings of existing applications. By utilizing modern technologies such as Flutter for frontend development and Supabase for backend services, the system ensures scalability, performance, and real-time data synchronization. The integration of locally relevant payment methods enhances accessibility for users, while a simplified and intuitive interface improves overall usability. Additionally, the inclusion of an efficient admin panel allows better control over menu management and order processing, reducing delays and minimizing errors. Through this approach, City Sizzle not only aligns with current technological trends but also introduces enhancements that contribute to a more reliable and user-centered food delivery experience.

1.3.1 Analysis from Literature Review

The literature review of existing food delivery systems provides a strong foundation for analyzing current trends, identifying system limitations, and defining the direction for the proposed solution. A careful examination of widely used platforms such as Foodpanda, Uber Eats, and DoorDash reveals that although these applications have successfully digitized the food ordering process, they still exhibit several structural, operational, and user-experience-related shortcomings. This analysis not only highlights the gaps in existing systems but also justifies the need for a more refined and context-aware solution such as City Sizzle.

One of the most critical observations from the literature is the inconsistency in real-time system performance. While modern applications claim to provide live order tracking and instant updates, in practice, there are frequent delays in reflecting accurate order statuses, particularly during peak hours. This issue arises due to inefficient synchronization between the frontend interface, backend processing, and database updates. As a result, users often receive delayed confirmations or outdated order information, which directly impacts trust and satisfaction. From this analysis, it becomes evident that real-time data handling must be treated as a core system requirement rather than an optional feature. The proposed system, therefore, emphasizes the use of a real-time backend service to ensure immediate updates and consistent system behavior. Another important aspect identified through the literature is the gap in localized payment integration. Although global platforms support a wide range of payment methods, they often lack proper integration with region-specific digital wallets. In countries like Pakistan, services such as JazzCash and EasyPaisa dominate the digital payment landscape, yet their implementation in many systems is either limited or not fully optimized. This creates inconvenience for users who rely on these services for everyday transactions. The analysis clearly indicates that effective payment integration must be tailored to the target user base, ensuring both accessibility and security. Consequently, City Sizzle incorporates these local payment gateways as a central feature rather than an additional option, thereby improving usability and user adoption.The study also reveals that user interface design plays a significant role in the overall effectiveness of food delivery applications. While some platforms offer visually appealing interfaces, they often compromise simplicity and ease of navigation. Complex layouts, excessive information, and multi-step ordering processes can confuse users and increase the time required to complete a transaction. This issue is particularly relevant for new users who may not be familiar with the application structure. From this observation, it is clear that usability should be prioritized over visual complexity. The proposed system

addresses this by focusing on a clean, intuitive, and minimalistic design that reduces user effort and enhances interaction efficiency.

In addition to user-side challenges, the literature highlights several limitations in administrative control and backend management. Many existing systems do not provide sufficient tools for administrators to manage menu items, update order statuses, or monitor system performance in real time. This lack of control leads to operational inefficiencies, such as outdated menu listings and delayed order processing. Furthermore, the absence of structured data management practices can result in inconsistencies and redundancy within the database. The analysis suggests that a welldesigned admin panel and a properly structured database are essential for maintaining system reliability. City Sizzle addresses these issues by implementing a dedicated administrative interface and a relational database structure that ensures data accuracy and consistency.

Another key insight derived from the literature is the issue of cost and affordability. High delivery charges and service fees in existing platforms often discourage frequent usage, especially among price-sensitive users. This indicates that system efficiency should not only focus on technical performance but also on optimizing operational costs. By streamlining processes and reducing unnecessary overhead, a system can provide better value to its users. The proposed solution takes this into consideration by designing a lightweight and efficient architecture that minimizes resource consumption and operational complexity.

From a technological perspective, the analysis confirms that modern frameworks and backend services provide powerful tools for building scalable and responsive applications. However, the effectiveness of these technologies depends on their proper integration and usage. Poor architectural decisions can lead to performance issues, security vulnerabilities, and limited scalability. Therefore, the proposed system adopts a structured architectural approach that separates concerns between the user interface, business logic, and data management layers. This ensures that the system remains maintainable, flexible, and capable of handling future enhancements.

In conclusion, the analysis of the literature clearly demonstrates that while existing food delivery systems have achieved significant progress, they still fall short in areas such as real-time performance, localized payment integration, usability, administrative control, and cost

efficiency. These findings directly influence the design and development of City Sizzle, guiding it toward a more user-centered and technically robust solution. By addressing the identified gaps and leveraging modern technologies effectively, the proposed system aims to provide a more reliable, efficient, and accessible food ordering experience that meets the evolving needs of users.

1.4 Methodology and Software Development Lifecycle

The development of City Sizzle is based on the **Agile Model** [5], which is widely recognized for its flexibility, iterative nature, and focus on continuous improvement. Unlike traditional linear development models, Agile allows the system to evolve gradually through repeated development cycles, making it particularly suitable for mobile application projects where requirements may change over time.

In the context of City Sizzle, Agile enables the development team to divide the project into smaller functional components and implement them in stages. Each stage, commonly referred to as a sprint, focuses on a specific feature such as user authentication, menu browsing, cart management, or payment processing. After the completion of each sprint, the developed module is tested and refined based on feedback, ensuring that errors are identified early and improvements can be made without affecting the entire system.This methodology promotes better collaboration, faster delivery of working features, and improved adaptability to changes. It also reduces development risks by allowing continuous validation of system functionality throughout the project lifecycle.

1.5 Software Development Lifecycle Overview

The software development lifecycle (SDLC) [6] of City Sizzle consists of several structured phases, each contributing to the successful completion of the project. These phases are interconnected and often overlap due to the iterative nature of Agile development.

1.5.1 Requirement Gathering and Analysis

The lifecycle begins with the requirement gathering phase, where the needs of the system are identified and documented. This phase involves understanding user expectations, defining system functionalities, and determining the scope of the application. All requirements are formally documented in the Software Requirements Specification (SRS), which serves as a reference throughout the development process.

The system requirements include core features such as user registration, menu browsing, order placement, payment processing through JazzCash and EasyPaisa, and an admin panel for system management. In addition to functional requirements, non-functional aspects such as performance, security, and usability are also considered. Proper analysis at this stage ensures that the system is designed with a clear and well-defined objective.

1.5.2 System Design Phase

After defining the requirements, the project moves into the design phase, where the overall structure and architecture of the system are developed. This phase is documented in the Software Design Document (SDD), which provides a detailed blueprint for system implementation.

The design phase includes defining the system architecture, database structure, and interaction between different components. The application is designed using a modular approach, separating the frontend, backend, and database layers. This separation ensures better maintainability and scalability. Special consideration is given to designing a responsive user interface, efficient data flow, and secure integration of payment systems.Various design models and diagrams are prepared during this phase, including use case diagrams, class diagrams, and data flow diagrams, which help in visualizing the system structure and functionality.

1.5.3 Implementation Phase

The implementation phase involves the actual development of the system based on the design specifications. In City Sizzle, this phase includes building the mobile application using Flutter and integrating it with the Supabase backend. The system is developed in modules to ensure better organization and easier debugging. These modules include authentication, menu management, cart system, order processing, payment integration, and admin functionalities. Each module is coded, tested individually, and then integrated into the main system. The modular approach ensures that any issues can be isolated and resolved without affecting other parts of the application.

1.5.4 Testing Phase

Testing is an essential part of the development lifecycle and is carried out continuously throughout the project. The purpose of testing is to ensure that the system functions correctly

and meets all specified requirements. Different types of testing are performed, including unit testing to verify individual components, integration testing to ensure proper interaction between modules, and system testing to validate the complete application. Special attention is given to critical components such as payment processing, order confirmation, and database operations. Errors identified during testing are corrected in subsequent iterations, ensuring continuous improvement of the system.

1.5.5 Deployment Phase

Once the system has been fully developed and tested, it is prepared for deployment. This phase involves configuring the application for real-world use, ensuring that all components are properly integrated and functioning as expected. The backend services are deployed using Supabase, and the mobile application is prepared for installation on Android devices. Performance optimization and security checks are also conducted to ensure a smooth user experience.

1.5.6 Maintenance and Future Enhancements

After deployment, the system enters the maintenance phase, where it is continuously monitored and updated. Any issues reported by users are addressed, and necessary improvements are implemented.

This phase also includes adding new features and enhancements based on user feedback and technological advancements. The use of Agile methodology supports continuous updates, making it easier to expand the system in the future without major restructuring.

1.5.7 Rationale for Selecting Agile Methodology

The selection of Agile as the development methodology is justified by several factors. The iterative nature of Agile allows the system to adapt to changing requirements, which is common in mobile application development. It also enables early detection of errors, reducing the risk of major issues during later stages of the project. Furthermore, Agile promotes continuous feedback and improvement, ensuring that the final product aligns closely with user expectations. This makes it an ideal choice for developing a dynamic and user-centered application like City Sizzle.

1.6 Object-Oriented Methodology in City Sizzle

In addition to the Agile development approach, the City Sizzle system is designed and implemented using the principles of **Object-Oriented Programming (OOP)**. This methodology focuses on structuring the system as a collection of interacting objects, each representing a real-world entity within the application. The use of object-oriented concepts plays a significant role in improving code organization, reusability, scalability, and maintainability throughout the development process.

In the context of City Sizzle, the system is naturally modeled around real-world entities such as users, food items, orders, carts, and payments. Each of these entities is represented as a class in the system, with its own attributes (data) and methods (functions). For example, a user object may contain attributes such as name, email, and password, along with methods for login and registration, while an order object includes details such as order ID, status, and total price, along with methods for placing and updating orders. This object-based representation makes the system more intuitive and easier to manage.

The application of object-oriented principles ensures that the system components are logically organized and loosely coupled. Instead of writing repetitive or tightly connected code, the system is divided into independent modules that can interact with each other through well-defined interfaces. This not only simplifies development but also makes it easier to modify or extend the system in the future.

The following core OOP concepts are applied within the City Sizzle project:

- **Encapsulation:**
  Data and related methods are grouped together within classes, ensuring that internal details are hidden from other parts of the system. This improves data security and prevents unintended modifications.
- **Inheritance:**
  Common functionalities can be shared among different classes, reducing code duplication. For example, different types of users (such as admin and customer) can inherit common properties from a base user class.
- **Polymorphism:**

The system allows methods to behave differently based on the object that invokes them. This flexibility is useful when handling different types of operations, such as multiple payment methods.

- **Abstraction:**

  Complex system details are hidden, and only essential functionalities are exposed to the user. This simplifies interaction with the system and improves overall usability.

The use of object-oriented methodology is further supported by Flutter, which is based on the Dart programming language and strongly encourages the use of classes and reusable components. In Flutter, UI elements themselves are treated as objects (widgets), which aligns well with object-oriented design principles. This allows the developer to create reusable UI components, manage application state effectively, and maintain a clean code structure.

Overall, the adoption of object-oriented methodology in City Sizzle enhances the quality of the system by promoting modular design, reducing complexity, and enabling easier maintenance. It ensures that the application remains scalable and adaptable to future changes, making it a robust solution for modern food ordering systems.

CHAPTER 2

PROBLEM DEFINATION

2 Problem Definition (Overview)

The purpose of this chapter is to clearly define the problem that the proposed system, City Sizzle, aims to solve. A well-defined problem statement is essential for the successful development of any software system, as it provides a clear direction for system design, implementation, and evaluation.In the context of modern digital services, the food delivery industry has witnessed rapid growth due to increasing consumer demand for convenience and efficiency. However, despite the availability of various food ordering platforms, several issues continue to affect system performance and user satisfaction. These issues highlight the need for an improved solution that can address the shortcomings of existing systems.

City Sizzle is designed to overcome these limitations by providing a more efficient, reliable, and user-friendly food ordering platform that integrates modern technologies and secure payment systems

2.1 Problem Statement

The food delivery industry has undergone significant digital transformation over the past decade, driven by increasing consumer demand for convenience, speed, and accessibility. Online food ordering systems have become an essential part of modern lifestyle, allowing users to place orders from restaurants without physical interaction. Despite this advancement, most existing food delivery platforms still do not fully meet the expectations of users and administrators due to several persistent functional, technical, and usability-related issues. These limitations reduce the overall efficiency of such systems and highlight the need for a more integrated and optimized solution like City Sizzle.

One of the most critical problems in existing systems is the lack of real-time synchronization across different modules of the application. In many current platforms, the communication between customers, restaurants, and delivery management systems is not fully instantaneous. As a result, users often experience delays in order confirmation, inaccurate updates regarding order status, and incomplete visibility of the order lifecycle. For instance, a customer may place an order and receive delayed confirmation, or in some cases, the system may fail to reflect the current status of preparation or dispatch in real time. This lack of transparency creates uncertainty and reduces user trust in the system.

Another major issue is the inefficient handling of user experience and interface design. Many existing food delivery applications are either overly complex or poorly optimized for smooth

navigation. Users, especially first-time users, often face difficulty in browsing menus, selecting items, customizing orders, and completing checkout processes. The absence of a clean, intuitive, and responsive user interface leads to confusion and increases the time required to complete simple tasks. Additionally, inconsistent design across different devices further worsens the usability experience, making the system less reliable for a diverse user base.

Performance issues also play a significant role in reducing the effectiveness of current systems. During peak hours, such as lunch and dinner times, many platforms experience heavy traffic loads that result in slow response times, delayed loading of menus, and sometimes complete system crashes. These performance bottlenecks are usually caused by poor backend optimization, lack of proper load balancing, and inefficient database queries. As user demand continues to grow, these limitations become more severe, making it difficult for existing systems to scale effectively.

Payment processing is another area where existing food delivery systems show considerable weaknesses. Although some platforms support digital payment methods, there is often limited integration with region-specific and widely used payment gateways such as JazzCash and EasyPaisa. This restricts accessibility for a large portion of users who rely on mobile-based payment solutions. Furthermore, some systems do not provide secure or reliable transaction handling, leading to failed payments, duplicate charges, or lack of proper payment confirmation. Inadequate encryption and weak security protocols also increase the risk of data breaches and unauthorized access to sensitive financial information.From an administrative perspective, current systems often lack efficient management tools and automation features. Administrators are frequently required to manually update menus, manage orders, and track system activity, which increases the chances of human error and delays in processing. In many cases, there is no centralized dashboard that provides a real-time overview of system operations, making it difficult for administrators to monitor performance or respond quickly to issues. The absence of advanced analytics and reporting tools further limits decision-making capabilities, preventing effective optimization of services.

Database management and system architecture also contribute to the overall inefficiency of existing solutions. Many platforms are built on outdated or poorly structured database systems

that lead to data redundancy, inconsistency, and slow query performance. Additionally, lack of proper API integration and modular system design makes it difficult to introduce new features or scale the system according to growing demands. This results in rigid systems that are not adaptable to modern technological requirements.Security concerns remain one of the most serious issues in current food delivery systems. Many applications do not implement strong authentication mechanisms, secure session handling, or proper data encryption techniques. This exposes users to potential risks such as unauthorized access, data leakage, and manipulation of order or payment information. In an era where digital transactions are increasing rapidly, such vulnerabilities significantly reduce user confidence in the platform.Another underlying issue is the lack of effective communication channels within the system. Users are often unable to directly interact with administrators or restaurants in real time, which leads to delays in resolving queries, complaints, or order-related issues. Notification systems in many platforms are also limited or inconsistent, meaning users do not always receive timely updates about their orders. This weak communication structure further contributes to dissatisfaction and inefficiency.

Considering all these challenges collectively, it becomes evident that existing food delivery systems are not fully optimized to deliver a seamless, secure, and efficient user experience. They lack proper integration between core modules such as ordering, payment processing, real-time tracking, and administrative control. These gaps create operational inefficiencies, reduce scalability, and negatively impact user satisfaction.

Therefore, the fundamental problem addressed in this project is the absence of a comprehensive, real-time, and fully integrated food ordering system that can efficiently manage customer interactions, ensure secure and flexible payment processing, provide accurate order tracking, and support effective administrative control within a scalable and user-friendly environment. The City Sizzle system is proposed to resolve these issues by introducing a modern architecture that focuses on performance optimization, usability, security, and real-time system coordination.

2.2 Deliverables of the System

The City Sizzle project is expected to produce a complete and functional food ordering and delivery management system that fulfills both user and administrative requirements. The deliverables of this system represent the final outputs that will be provided after successful

development, testing, and implementation of the project.The primary deliverable of this project is a fully functional mobile application developed using Flutter that allows customers to browse food items, place orders, make secure payments, and track their orders in real time. The application will be designed with a responsive and user-friendly interface to ensure smooth interaction across different Android devices. The customer application will include essential features such as menu browsing, item selection, cart management, order placement, payment integration, and order status tracking.Another major deliverable is the administrative panel, which will enable system administrators to manage the entire food ordering process efficiently. This panel will provide functionalities such as adding, updating, and removing menu items, managing incoming orders, monitoring order status, and handling customer queries. The admin interface will also support realtime updates, allowing administrators to maintain accurate and updated system information at all times.

A secure backend system integrated with Supabase is also a key deliverable of the project. This backend will handle database management, authentication, real-time data synchronization, and API communication between the frontend application and the database. It ensures that all user actions, such as placing orders or making payments, are processed efficiently and stored securely.

In addition to the software components, proper system documentation is also an important deliverable. This includes the Software Requirements Specification (SRS), system design diagrams, database schema, testing reports, and user manuals. These documents will provide complete technical and functional details of the system and will assist in future maintenance or enhancements.

Overall, the final deliverables of City Sizzle include:

- A fully functional Flutter-based mobile application for customers
- A web-based or integrated admin panel for system management
- A secure and scalable backend using Supabase
- Complete system documentation and technical reports
- Testing and validation reports ensuring system reliability

2.2.1 Development Requirements

The development of the City Sizzle system requires a combination of hardware, software, and technical tools to ensure smooth design, implementation, and testing of the application. These requirements define the environment in which the system will be developed and deployed.

2.2.2 Hardware Requirements

The hardware requirements for developing and running the system are minimal but essential to ensure efficient performance during development and testing phases. A standard development machine is required that supports modern software development tools. The system should include a computer or laptop with at least a dual-core processor or higher to handle development tasks efficiently. A minimum of 8 GB RAM is recommended to ensure smooth operation of Flutter development tools and emulators. Sufficient storage space is also required to store project files, dependencies, and development resources. Additionally, an Android smartphone or emulator is necessary for testing the mobile application in a real or simulated environment. A stable internet connection is also required for accessing backend services, APIs, and cloud-based resources.

2.2.3 Software Requirements

The software requirements consist of tools, frameworks, and platforms used for developing both the frontend and backend of the system. The mobile application will be developed using Flutter, which allows cross-platform development and ensures consistent performance across different devices. The Dart programming language will be used for implementing application logic within Flutter. For backend development, Supabase will be used as the primary platform. It provides database management, authentication services, real-time data synchronization, and API support, which are essential for building a scalable and responsive system. A code editor such as Visual Studio Code will be used for writing and managing the project code efficiently. Git will be used for version control to track changes in the project and maintain code history.In addition, Android Studio may be used for running and testing Android emulators during development. API integration tools and JSON handling libraries will also be required for communication between frontend and backend systems.

2.2.4 Programming and Framework Requirements

The system development requires knowledge of specific programming languages and frameworks. Flutter and Dart will be used for frontend development, while Supabase will handle backend services. RESTful APIs will be used for communication between different system components. Firebase or similar services may also be optionally integrated for notifications or additional features. Understanding of database design principles is also necessary to ensure efficient data storage and retrieval

2.2.5 Network Requirements

Since City Sizzle is an online food ordering system, a stable and continuous internet connection is required for both development and usage. The system relies heavily on real-time communication between users and the backend server, making network reliability an essential requirement. Cloudbased services such as Supabase require uninterrupted connectivity for proper functioning.

2.2.6 Human and Technical Requirements

The development of the system also requires skilled technical knowledge in mobile application development, backend integration, and database management. The developer must have understanding of UI/UX design principles to create a user-friendly interface. Problem-solving skills are also necessary to handle errors, optimize performance, and ensure smooth system operation.

2.3 Current System (Existing Food Delivery Systems)

The current food delivery systems available in the market are widely used and have significantly improved the way customers interact with restaurants and food services. These systems typically operate through mobile applications or web platforms that connect users with multiple restaurants, allowing them to browse menus, place orders, and receive food at their doorstep. Popular platforms such as Foodpanda, Uber Eats, and local restaurant-based ordering systems have made food ordering more convenient than traditional manual methods. However, despite their popularity and widespread usage, these systems still exhibit several functional and technical limitations that reduce their overall efficiency and user satisfaction.

In the existing systems, the general workflow begins when a user opens the application, selects a restaurant, browses available food items, adds items to a cart, and proceeds to checkout for payment. Once the order is placed, it is forwarded to the restaurant, where it is manually or semiautomatically processed. The restaurant then confirms the order, prepares the food, and dispatches it for delivery. Although this process appears simple, in practice it is often affected by delays, communication gaps, and lack of synchronization between different system components.

One of the major characteristics of current systems is their dependency on third-party integrations and centralized servers. While this enables large-scale operations, it also introduces performance bottlenecks and reduces flexibility in customization. Many systems rely heavily on external APIs for payment processing, mapping, and delivery tracking, which can lead to inconsistencies if those services experience downtime or slow response times.

Another important aspect of the current system is that it primarily focuses on customer-facing functionality while providing limited optimization for administrative operations. Restaurant owners and system administrators often have restricted control over real-time updates, order prioritization, and system-wide monitoring. This results in delayed updates, inefficient order handling, and reduced operational transparency. In most existing platforms, real-time communication between customers, restaurants, and delivery personnel is either limited or partially implemented. Although some systems provide order tracking features, these updates are not always accurate or instantaneous. Users may experience situations where the displayed order status does not match the actual progress of food preparation or delivery, which leads to confusion and dissatisfaction. Payment systems in current solutions are also partially optimized. While international payment methods such as credit/debit cards and digital wallets are supported, integration with region-specific mobile payment systems is often limited. This restricts accessibility for users in developing regions where mobile wallets like JazzCash and EasyPaisa are more commonly used. Additionally, issues such as failed transactions, delayed confirmations, and lack of proper refund mechanisms are frequently reported in existing systems.

From a technical perspective, many current systems face scalability challenges. During peak usage times, such as weekends or meal hours, servers often become overloaded, resulting in slow performance, application crashes, or failed order submissions. This indicates a lack of

efficient load balancing and optimized backend architecture. Furthermore, database structures in some systems are not designed efficiently, leading to redundancy, slow query execution, and data inconsistency.

User interface design is another area where existing systems show limitations. Although modern applications have improved visually, many still suffer from complex navigation, cluttered layouts, or inconsistent design patterns across different screens. This makes it difficult for users to complete tasks quickly and efficiently, especially for elderly users or those with limited technical experience. Security is also a concern in current food delivery systems. While basic authentication is implemented, advanced security measures such as strong encryption, secure session management, and multi-layer protection are not always fully enforced. This exposes systems to potential risks such as unauthorized access, data leaks, and payment vulnerabilities.

Overall, the current system can be summarized as a partially efficient but imperfect solution that provides basic food ordering functionality while still lacking in several critical areas such as realtime processing, scalability, secure payment integration, administrative control, and user experience optimization. These limitations create a gap between user expectations and actual system performance. Due to these shortcomings, there is a clear need for an improved system that can integrate all essential functionalities into a single, efficient, and scalable platform. This gap in existing solutions directly leads to the development of the proposed City Sizzle system, which aims to overcome these issues by introducing better architecture, real-time processing, enhanced security, and improved usability.

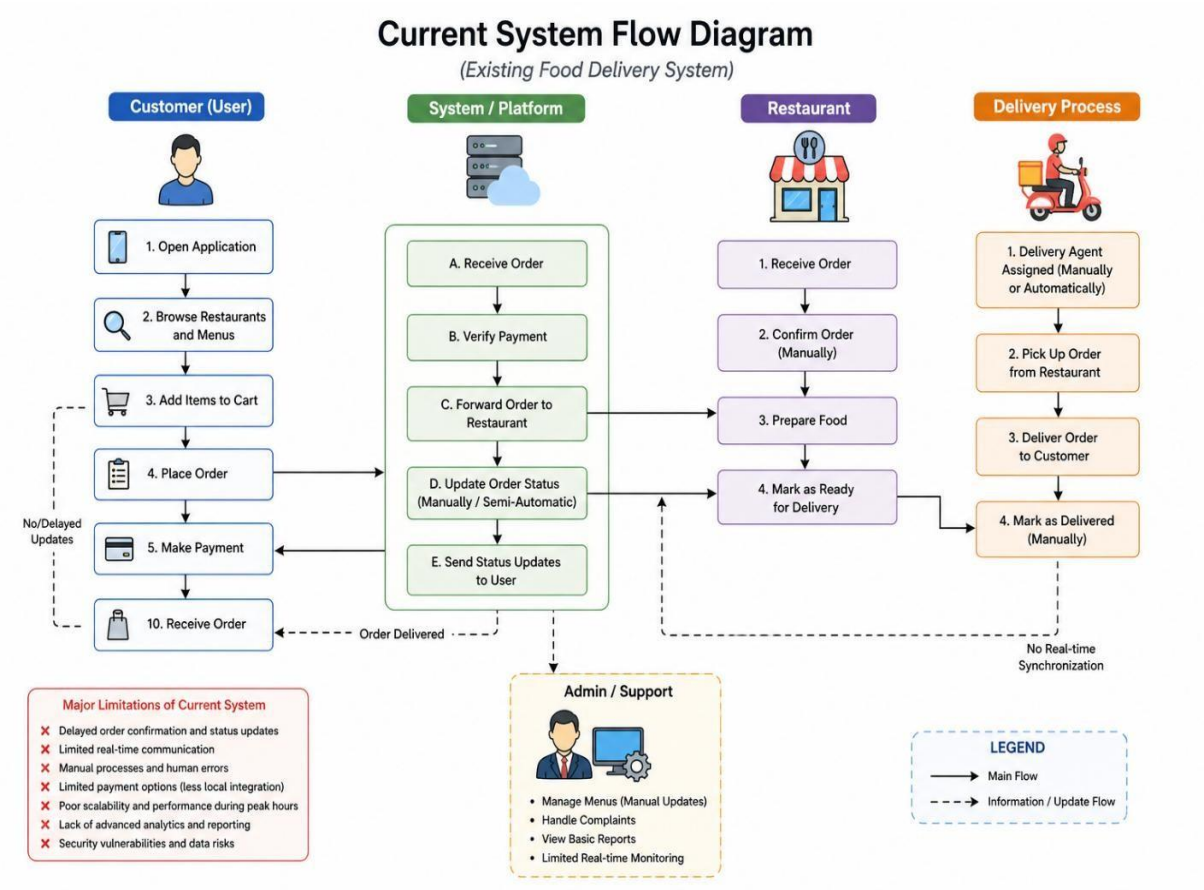

Figure 2.1: System Flow Diagram

CHAPTER 3

REQUIREMENTS ANALYSIS

3 Requirement Analysis

Requirement analysis is one of the most critical phases in the software development lifecycle, as it defines what the system is expected to do and under what constraints it must operate. In the context of the City Sizzle application, requirement analysis involves identifying the needs of both end users (customers) and administrators, and translating those needs into clear functional and non-functional requirements. This phase ensures that the developed system aligns with user expectations and solves the identified problems effectively.

The requirement analysis for City Sizzle was carried out through understanding existing food delivery systems, analyzing their limitations, and determining the essential features required to build an improved and efficient solution. The requirements were categorized into functional and non-functional requirements to provide a structured understanding of the system behavior.

3.1 Use case Diagrams

A use case diagram is a simple visual diagram used in software engineering to show what a system does and who uses it.

It is part of the Unified Modeling Language [10] (UML) and mainly focuses on the functional view of a system (not internal coding or database structure).

A use case diagram shows:

- Actors (users or external systems)
- Use cases (functions/features of the system)
- Relationship between them

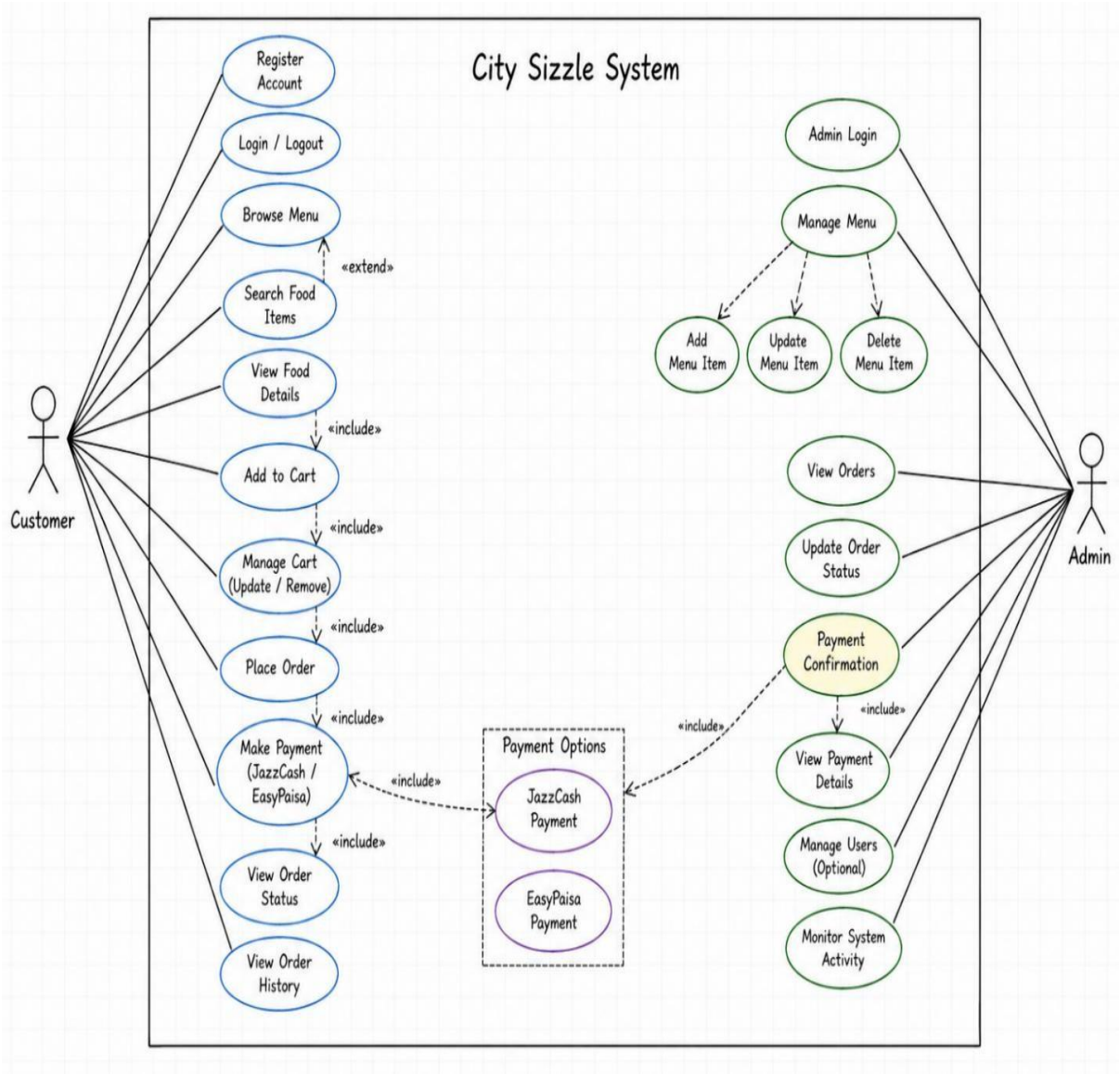

Figure 3.1: Use Case Diagram

3.1.1 UC-U1: User Registration

This use case describes the process of creating a new user account in the City Sizzle system. It ensures proper user identification and enables access to personalized features.

Table 3.1: User Registration

Field	**Description**

Use Case	User Registration
Actors	Customer
Description	This use case allows a new user to create an account in the City Sizzle system. The registration process is essential for identifying users uniquely and enabling personalized services such as order tracking, cart management, and payment processing.
Trigger	The use case starts when the user selects the “Sign Up” or “Create Account” option from the application interface.
Preconditions	The user must not already have an existing account in the system. The application must be connected to the internet to communicate with the Supabase backend.
Postconditions	A new user account is successfully created and stored in the database. The user profile is initialized and can now access system features after login.
Normal Flow	1. User opens the City Sizzle application. 2. User selects the “Sign Up” option. 3. System displays registration form. 4. User enters details (name, email, password, phone).

	5. System validates input fields. 6. System checks if email already exists. 7. If valid, system creates account in Supabase. 8. Confirmation message is displayed. 9. User is redirected to login screen.
Alternative Flow	- If invalid input is entered, system highlights errors. - If email exists, system asks user to login instead. - User may cancel registration anytime.
Exceptions	- Database connection failure - Invalid email format or weak password - Server timeout - Duplicate email error
Business Rules	- Email must be unique - Password must meet security standards - User must be verified before full access (if enabled)
Assumptions	- User has valid email - Internet connection is available

	- Supabase backend is working

3.1.2 UC-U2: Login / Logout

This use case handles secure authentication for users to access and exit the system. It manages session creation and termination for safe application usage.

Table 3.2: User Login & Logout

Field	**Description**
Use Case	User Login and Logout
Actors	Customer
Description	This use case allows registered users to securely access their accounts and log out when finished. It ensures authentication and session management.
Trigger	Login starts when user enters credentials. Logout starts when user clicks logout button.
Preconditions	User must already be registered for login. User must be logged in for logout.
Postconditions	Login creates a session. Logout destroys session and clears authentication data.

Normal Flow	1. User opens login screen. 2. Enters email and password.
	3. System validates credentials. 4. If correct, session is created. 5. User is redirected to dashboard. 6. User clicks logout. 7. System clears session. 8. User redirected to login screen.
Alternative Flow	- Forgot password option is available. - Persistent login may keep user signed in.
Exceptions	- Incorrect credentials - Account not found - Network failure - Session expired

Business Rules	- Only registered users can log in - Sessions must expire for security - Multiple failed attempts may lock account
Assumptions	- Credentials exist in Supabase - Authentication service is active

3.1.3 UC-U3: Browse Food Menu

This use case allows users to view all available food items categorized in the menu. It ensures smooth browsing of updated and organized food listings.

Table 3.3: Browse Food Menu

Field	Description
Use Case	Browse Food Menu
Actors	Customer
Description	This use case allows users to view all available food items categorized into sections such as burgers, pizzas, drinks, and meals.
Trigger	User opens the menu section in the app.
Preconditions	User must be logged in and menu data must exist in database.

Postconditions	Food items are displayed successfully.
Normal Flow	1. User opens menu screen. 2. System fetches data from Supabase. 3. Items are grouped into categories. 4. Items are displayed with image, price, description. 5. User scrolls through menu.
Alternative Flow	- User filters items by category. - User refreshes menu list.
Exceptions	- Server failure - Image loading error
Business Rules	- Only available items are shown - Out-of-stock items are hidden or marked
Assumptions	- Menu is properly stored in database

3.1.4 UC-U4: View Food Item Details

This use case provides detailed information about a selected food item. It helps users make informed decisions before placing an order.

Table 3.4: Food Item Details

Field	Description
Use Case Name	View Food Item Details
Actors	Customer
Description	This use case allows users to view complete information of a selected food item including image, description, price, ingredients (if available), and availability status. It helps users make informed decisions before ordering.
Trigger	User taps on any food item from menu screen.
Preconditions	Menu must be loaded and user must be logged in.
Postconditions	Detailed view of selected food item is displayed.
Normal Flow	1. User selects a food item. 2. System retrieves item details from database. 3. System displays image, description, price. 4. User reviews item information.
Alternative Flow	- User goes back to menu without selecting item.

Exceptions	- Item data not found - Image fails to load
Business Rules	- Only active items can be viewed - Price must match database value
Assumptions	- Item details exist in Supabase

3.1.5 UC-U5: Manage Cart

This use case enables users to add, remove, or update food items in their cart. It ensures proper calculation and management of the selected order items.

Table 3.5: Manage Cart

Field	Description
Use Case Name	Manage Cart
Actors	Customer
Description	This use case allows users to modify their cart before placing an order, including increasing/decreasing quantity or removing items completely.
Trigger	User opens cart screen.

Preconditions	Cart must contain at least one item.
Postconditions	Updated cart is saved and total amount recalculated.
Normal Flow	1. User opens cart. 2. System displays selected items. 3. User increases/decreases quantity. 4. User removes items if needed. 5. System updates total bill.
Alternative Flow	- User clears entire cart
Exceptions	- Item removed from menu while in cart
Business Rules	- Quantity must be at least 1 - Total must update instantly
Assumptions	- Cart stored locally or in database

3.1.6 UC-U6: Process Payment

This use case handles secure online payment through integrated gateways like JazzCash or EasyPaisa. It ensures successful transaction processing before order confirmation.

Table 3.6: Payment Process

Field	**Description**
Use Case Name	Process Payment
Actors	Customer, Payment Gateway
Description	Handles secure online payment using JazzCash or EasyPaisa for placed orders.
Trigger	User selects payment method after placing order.
Preconditions	Order must exist and payment must be pending.
Postconditions	Payment status updated (Success/Failed).
Normal Flow	1. User selects payment method. 2. System redirects to gateway. 3. User enters payment details. 4. Gateway verifies transaction. 5. Payment confirmation received.

Alternative Flow	- User retries payment - User changes payment method
Exceptions	- Insufficient balance - Gateway timeout - Transaction failure
Business Rules	- Order is confirmed only after successful payment
Assumptions	- Payment APIs are active

3.1.7 UC-U7: View Order History & Track Order

This use case allows users to view past orders and track current order status. It provides real-time updates on order progress.

Table 3.7 : Order History & Tracking

Field	**Description**
Use Case Name	Order History & Tracking
Actors	Customer
Description	Allows users to view past orders and track current order status in real-time (Pending, Preparing, Delivered).

Trigger	User opens “My Orders” section.
Preconditions	User must be logged in.
Postconditions	Order history and status displayed.
Normal Flow	1. User opens order history. 2. System fetches user orders. 3. Orders displayed with status. 4. User selects order to view details.
Alternative Flow	- Filter orders by date/status
Exceptions	- No orders found
Business Rules	- Users can only see their own orders
Assumptions	- Orders stored in database

3.1.8 UC-U8: Cancel Order

This use case allows users to cancel an order before it is processed. It ensures only pending orders can be safely removed from the system.

Table 3.8: Order Cancellation

Field	Description
Use Case Name	Cancel Order
Actors	Customer
Description	Allows users to cancel an order before it is processed or prepared by admin.
Trigger	User clicks “Cancel Order” option.
Preconditions	Order must be in Pending state.
Postconditions	Order status updated to Cancelled.
Normal Flow	1. User opens order details. 2. Clicks cancel order. 3. System verifies order status.
	4. Order is cancelled successfully.
Alternative Flow	- If order already preparing, cancellation denied
Exceptions	- Order already delivered

Business Rules	- Only pending orders can be cancelled
Assumptions	- Admin may receive cancellation notification

3.1.9 UC-A1: Admin Login

This use case enables administrators to securely access the admin dashboard. It ensures only authorized users can manage system operations.

Table 3.9: Admin Login

Field	**Description**
Use Case Name	Admin Login
Actors	Admin
Description	This use case allows administrators to securely access the admin dashboard for managing system operations such as orders, users, and menu.
Trigger	Admin enters login credentials.
Preconditions	Admin account must already exist.
Postconditions	Admin dashboard is successfully accessed.

Normal Flow	1. Admin opens login page. 2. Enters credentials. 3. System verifies admin role. 4. Access is granted. 5. Dashboard is displayed.
Alternative Flow	- Admin may reset password. - Admin retries login if failed.
Exceptions	- Invalid credentials - Unauthorized access attempt
Business Rules	- Only admin role can access dashboard - Admin login is more secure than user login
Assumptions	- Admin credentials are pre-registered

3.1.10 UC-A2: View Orders

This use case allows admin to view all customer orders in the system. It supports order monitoring and management efficiently.

Table 3.10: View Order

Field	Description
Use Case Name	View Orders
Actors	Admin
Description	Admin can view all customer orders for monitoring and management purposes.
Trigger	Admin opens orders panel.
Preconditions	Admin must be logged in.
Postconditions	Orders are displayed successfully.
Normal Flow	1. Admin opens order section. 2. System fetches all orders. 3. Orders are displayed with details. 4. Admin selects specific order.
Alternative Flow	- Filter orders by status
	- Search order by ID

Exceptions	- Database failure - No orders found
Business Rules	- Admin can view all user orders - Order history must be stored permanently
Assumptions	- Orders are synced in real-time

3.1.11 UC-A3: Manage Menu

This use case enables admin to add, update, or delete food items in the menu. It ensures the menu remains accurate and up to date.

Table 3.11: Manage Food Menu

Field	**Description**
Use Case Name	Manage Food Menu
Actors	Admin
Description	Admin manages food items including adding, updating, and deleting items to keep menu updated.
Trigger	Admin opens menu management section.

Preconditions	Admin must be authenticated.
Postconditions	Menu is updated in real-time for users.
Normal Flow	1. Admin opens menu panel. 2. Adds new food item. 3. Updates existing item. 4. Deletes unavailable item. 5. System updates database.
Alternative Flow	- Admin disables item temporarily - Admin updates price during promotion
Exceptions	- Image upload failure - Invalid input data
Business Rules	- Every item must have name, price, image - Deleted items must not appear in user app
Assumptions	- Image storage is supported - Database is real-time

3.1.12 UC-A4: Add New Food Item

This use case allows admin to add new food items with complete details. It ensures new products are properly stored in the system.

Table 3.12: Update Food Menu

Field	**Description**
Use Case Name	Add Food Item
Actors	Admin
Description	Admin adds new food items to menu including name, price, category, and image.
Trigger	Admin clicks “Add Item”
Preconditions	Admin must be logged in
Postconditions	New item added to menu
Normal Flow	1. Admin opens menu panel. 2. Enters item details. 3. Uploads image. 4. Saves item. 5. System updates menu.

Exceptions	- Invalid data - Image upload failure
Business Rules	- Every item must have required fields
Assumptions	- Storage system supports images

3.1.13 UC-A5: Delete Food Item

This use case allows admin to remove unavailable or discontinued food items. It ensures only valid items are displayed to users.

Table 3.13: Delete Items

Field	Description
Use Case Name	Delete Food Item
Actors	Admin
Description	Admin removes food items that are unavailable or discontinued.
Trigger	Admin clicks delete button
Preconditions	Item must exist

Postconditions	Item removed from menu
Normal Flow	1. Admin selects item. 2. Clicks delete. 3. System confirms deletion.
Exceptions	- Item linked to active order
Business Rules	- Deleted items must not appear in menu
Assumptions	- Admin has permission

3.1.14 UC-A6: Manage Users

This use case enables admin to view and control registered user accounts. It includes actions like blocking or monitoring users.

Table 3.14: Manage Users

Field	**Description**
Use Case Name	Manage Users
Actors	Admin

Description	Admin views and manages registered users including blocking or monitoring accounts.
Trigger	Admin opens user panel
Preconditions	Admin must be logged in
Postconditions	User status updated if changed
Normal Flow	1. View user list. 2. Search user. 3. Block/unblock user.
Exceptions	- User not found
Business Rules	- Admin can manage all users
Assumptions	- User data stored in system

3.1.15 UC-A7: Monitor System Activity

This use case allows admin to track system logs, user actions, and order flow. It ensures smooth and secure system performance.

Table 3.15: System Monitoring

Field	Description
Use Case Name	System Monitoring
Actors	Admin
Description	Admin monitors system logs, user activity, and order flow to ensure smooth operation.
Trigger	Admin opens dashboard analytics
Preconditions	Admin logged in
Postconditions	System activity displayed
Normal Flow	1. View analytics dashboard. 2. Check logs. 3. Monitor orders/users.
Exceptions	- Data loading error
Business Rules	- Only admin can access logs
Assumptions	- Logging system active

3.1.16 UC-A8: Payment Confirmation

This use case enables admin to verify payment success or failure for orders. It ensures only paid orders are processed further.

Table 3.16: Payment Confirmation

Field	**Description**
Use Case Name	Payment Confirmation
Actors	Admin, Payment Gateway
Description	Admin verifies whether payments for orders are successful or failed.
Trigger	Payment received notification
Preconditions	Order must be placed
Postconditions	Payment status updated
Normal Flow	1. System receives payment response. 2. Admin verifies transaction. 3. Order marked as paid.
Exceptions	- Payment mismatch
Business Rules	- Order cannot proceed without payment

Assumptions	- Gateway response is reliable

3.2 Functional Requirements

Functional requirements define the core features and services that the system must provide to its users. For City Sizzle, these requirements focus on customer interaction, order processing, payment handling, and administrative control.

The main functional requirements of the system include:

- The system shall allow users to register and log in securely using authentication.
- The system shall enable users to browse available food items along with categories and descriptions.
- The system shall allow users to add food items to a cart and modify quantities before placing an order.
- The system shall enable users to place food orders successfully.
- The system shall provide secure payment integration using JazzCash and EasyPaisa.
- The system shall generate an order confirmation after successful payment or order placement.
- The system shall allow users to view their order history and current order status.
- The system shall allow administrators to add, update, and delete food items from the menu.
- The system shall allow administrators to view and manage all customer orders.
- The system shall provide real-time synchronization of data between the application and backend (Supabase).

These functional requirements ensure that both customer-side and admin-side operations are fully supported within the system.

3.2.1 FR 01: User Authentication

This requirement defines the authentication process of the system. It ensures that only valid and registered users can access the application. It also provides secure login and session management using Supabase authentication services.

Table 3.17: FR-01 (User Auth)

Field	**Description**
Requirement ID	FR-01
Requirement Name	User Authentication (Registration & Login)
Description	The system shall provide a secure authentication mechanism that allows new users to register by providing their personal details such as name, email, phone number, and password. It also allows registered users to log in securely using validated credentials. This ensures that every user in the system is uniquely identified and their data is protected. Authentication is handled using Supabase authentication services, which ensures encrypted and secure login sessions.
Actors	Customer
Priority	High
Input	User provides registration details (name, email, password, phone number) or login credentials (email and password).

Output	System creates a new user account or generates a secure login session token for authenticated access.
Functional Behavior	The system validates user input, checks email uniqueness, encrypts password data, and communicates with Supabase backend. Upon
	successful validation, it either creates a new user record or verifies login credentials and initiates a secure session.
Constraints	Email must be unique across system. Password must meet security requirements such as minimum length and complexity. System must maintain secure authentication using encrypted communication.
Preconditions	For registration: user must not already exist. For login: user must already be registered in the system.
Postconditions	A new user account is stored in database OR user is successfully logged into system with an active session.

3.2.2 FR-02: Browse Food Menu

This requirement allows users to explore the available food items offered in the system. It provides categorized menu display with images, descriptions, and pricing information, enabling users to easily select food items

Table 3.18: FR-02 (Browse Menu)

Field	**Description**

Requirement ID	FR-02
Requirement Name	Browse Food Menu
Description	The system shall allow users to browse a structured food menu that includes multiple categories such as burgers, pizzas, drinks, and meals. Each food item is displayed with relevant details such as name, image, description, and price. This feature enables users to explore available options before placing an order. The menu data is dynamically fetched from the Supabase database and updated in real-time based on admin changes.
Actors	Customer
Priority	High
Input	User requests menu view or category selection.
Output	System displays categorized list of food items with complete details.
Functional Behavior	The system retrieves food items from the backend database, organizes them into categories, and renders them in a user-friendly interface. Any updates made by the admin are automatically reflected in the menu.
Constraints	Only active and available food items are displayed. Out-of-stock or deleted items must not appear in the menu.
Preconditions	User must be logged into the system and internet connection must be active.

Postconditions	Food menu is successfully displayed to the user interface.

3.2.3 FR-03: Cart Management

This requirement defines how users manage selected food items before placing an order. It allows users to add items, update quantities, or remove items from the cart, ensuring flexibility before final order confirmation.

Table 3.19: FR-03 (Cart Management)

Field	**Description**
Requirement ID	FR-03
Requirement Name	Cart Management
Description	The system shall provide a cart management feature that allows users to add selected food items into a virtual cart before placing an order. Users can modify item quantities, remove items, or clear the cart entirely. The system automatically calculates the total price based on selected items and quantity changes, ensuring accurate billing before checkout.
Actors	Customer
Priority	High

Input	User-selected food items and quantity adjustments.
Output	Updated cart containing items and total calculated price.
Functional Behavior	The system stores selected items temporarily, updates quantity changes dynamically, and recalculates total price in real-time. Cart data is maintained per user session.
Constraints	Quantity must always be greater than zero. Cart must not contain unavailable items.
Preconditions	Food items must be available in the menu and user must be logged in.
Postconditions	Cart is updated and ready for order placement.

3.2.4 FR-04: Place Food Order

This requirement handles the core ordering functionality of the system. It allows users to confirm their selected cart items and place an order which is then stored in the backend system.

Table 3.20: FR-04 (Order Placing)

Field	**Description**
Requirement ID	FR-04

Requirement Name	Place Order
Description	The system shall allow users to place an order by confirming the items in their cart. Once the order is placed, the system generates a unique order record in
	the database with an initial status of “Pending”. This order is then forwarded to the admin dashboard for further processing.
Actors	Customer
Priority	High
Input	Final cart items and order confirmation request.
Output	Order confirmation and unique order ID.
Functional Behavior	The system validates cart contents, generates an order object, stores it in Supabase database, and assigns a default status.
Constraints	Cart must not be empty at the time of order placement.
Preconditions	User must be authenticated and cart must contain at least one item.

3.2.5 FR-05: Payment Processing

This requirement defines the payment system integration within City Sizzle. It ensures secure transactions using third-party payment gateways such as JazzCash and EasyPaisa.

Table 3.21: FR-05 (Payment Process)

Field	Description
Requirement ID	FR-05
Requirement Name	Payment Processing
Description	The system shall integrate secure digital payment gateways such as JazzCash and EasyPaisa to allow users to complete transactions. Payment processing ensures that the order is only confirmed after successful verification of the transaction from the payment provider.
Actors	Customer, Payment Gateway
Priority	High
Input	Payment method selection and transaction credentials.
Output	Payment success or failure response.

Functional Behavior	The system sends payment request to external gateway API, receives transaction response, and updates order payment status accordingly.
Constraints	Payment must be processed through secure and verified third-party services only.
Preconditions	Order must already exist in system before payment initiation.
Postconditions	Payment status is updated in database (Paid/Failed).

3.2.6 FR-06: Order Confirmation

This requirement ensures that every successful order is confirmed and acknowledged by the system. It improves user trust and provides order validation after placement or payment.

Table 3.22: FR-06 (Order Confirmation)

Field	Description
Requirement ID	FR-06
Requirement Name	Order Confirmation
Description	The system shall generate an automatic order confirmation once an order is successfully placed or payment is completed. This confirmation acts as proof that the system has received and recorded the order.

Actors	Customer
Priority	High
Input	Order details and payment status.
Output	Confirmation message with order ID.
Functional Behavior	The system validates order status and generates a confirmation response displayed to the user interface.
Constraints	Only successfully placed orders are confirmed.
Preconditions	Order must exist in database.
Postconditions	User receives confirmation of order placement.

3.2.7 FR-07: Order History & Tracking

This requirement allows users to track their current orders and view past order history. It improves transparency and helps users monitor their order progress.

Table 3.23:FR-07(Order Tracking & History)

Field	**Description**
Requirement ID	FR-07

Requirement Name	Order History & Tracking
Description	The system shall allow users to view their past orders along with real-time tracking of current orders. This includes order status updates such as Pending, Preparing, and Delivered, ensuring transparency in the ordering process.
Actors	Customer
Priority	Medium
Input	User ID for fetching order history.
Output	List of past and current orders with status.
Functional Behavior	System retrieves user-specific orders from database and displays them in chronological order with status updates.
Constraints	Users can only access their own order history.
Preconditions	User must be logged in.
Postconditions	Order history is displayed successfully.

3.2.8 FR-08: Manage Food Menu

This requirement defines the administrative control over food items. It allows the admin to maintain and update the restaurant.

Table 3.24: FR-08 (Menu Management)

Field	Description
Requirement ID	FR-08
Requirement Name	Manage Food Menu
Description	The system shall provide administrative functionality to manage food items, including adding new items, updating existing items, and removing unavailable items. These changes are reflected instantly in the customer-facing application.
Actors	Admin
Priority	High
Input	Food item details (name, price, image, category)
Output	Updated food menu in the system
Functional Behavior	Admin performs CRUD (Create, Read, Update, Delete) operations on menu items which are stored in Supabase and updated in real-time.
Constraints	Only admin can modify menu data
Preconditions	Admin must be authenticated
Postconditions	Menu is updated and visible to users immediately

3.2.9 FR-09: Manage Orders

This requirement enables the admin to control and monitor all customer orders. It ensures proper order lifecycle management within the system.

Table 3.25:FR-09(Order Manage)

Field	Description
Requirement ID	FR-09
Requirement Name	Order Management
Description	The system shall allow administrators to view, monitor, and manage all customer orders. The admin can update order status based on the processing stage and ensure a smooth order flow.
Actors	Admin
Priority	High
Input	Order data received from users
Output	Updated order status and tracking information
Functional Behavior	The admin accesses the order dashboard, reviews order details, and updates the order status accordingly
Constraints	Only the admin has permission to modify order status
Preconditions	Orders must exist in the system
Postconditions	Order status is updated and stored in the database

3.2.10 FR-10: Real-Time Data Synchronization

This requirement ensures that all system data remains updated in real-time across both frontend and backend systems. It improves consistency and user experience.

Table 3.26: FR-10(Real-Time Data Sync)

Field	**Description**
Requirement ID	FR-10
Requirement Name	Real-Time Synchronization
Description	The system shall ensure real-time synchronization between the frontend application and backend database (Supabase). Any updates made by users or admin are immediately reflected across the system without manual refresh.
Actors	System
Priority	High
Input	Database updates (orders, menu changes, user actions).
Output	Live updated UI across application.
Functional Behavior	The system continuously listens to database changes and automatically updates frontend interface in real-time.

Constraints	Requires stable internet connection.
Preconditions	Backend connection must be active.
Postconditions	Data consistency is maintained across system.

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system rather than specific behaviors. These requirements ensure that the City Sizzle application is reliable, efficient, and user-friendly.

The key non-functional requirements include:

- **Performance:** The system should respond quickly to user actions such as loading menus, placing orders, and processing payments.
- **Scalability:** The application should be able to handle an increasing number of users and orders without performance degradation.
- **Usability:** The user interface should be simple, intuitive, and easy to navigate for users of all technical backgrounds.
- **Security:** User data, authentication credentials, and payment information must be securely stored and transmitted.
- **Reliability:** The system should maintain high availability with minimal downtime, ensuring continuous service.
- **Maintainability:** The system architecture should support easy updates, debugging, and feature enhancements.
- **Compatibility:** The application should run smoothly on both Android and iOS devices using Flutter's cross-platform capabilities.

These non-functional requirements ensure that the system is not only functional but also practical and efficient in real-world usage.

3.4 User Requirements

User requirements describe what end users expect from the system in terms of functionality and experience. For City Sizzle, users mainly include customers and administrators.

Customer Requirements:

- Easy registration and login process
- Ability to view food menus with clear descriptions and prices
- Smooth ordering and cart management system
- Secure and fast payment options
- Order tracking and status updates
- Simple and attractive user interface

Admin Requirements:

- Dashboard to manage food items
- Ability to monitor and manage customer orders
- Control over menu updates (add/edit/delete items)
- View order history and system activity

3.5 System Requirements

System requirements define the technical environment needed to run the application effectively.

- **Frontend:** Flutter framework for mobile application development
- **Backend:** Supabase for authentication, database, and real-time updates
- **Database:** PostgreSQL (managed through Supabase)
- **Payment Integration:** JazzCash and EasyPaisa APIs
- **Development Tools:** Visual Studio Code, GitHub for version control
- **Platform Support:** Android and iOS devices

3.6 Assumptions and Constraints

During requirement analysis, certain assumptions and constraints were identified:

Assumptions:

- Users have access to smartphones and internet connectivity.
- Users are familiar with basic mobile application usage.
- Payment services (JazzCash/EasyPaisa) are available and functional.

Constraints:

- The system depends on third-party payment gateways.
- Internet connectivity is required for real-time functionality.
- Limited access to external restaurant APIs.

CHAPTER 4

DESIGN AND ARCHITECTURE

4 Design and Architecture

The design and architecture phase of the City Sizzle system focuses on defining how the system is structured and how different components interact with each other. This phase translates the functional requirements into a technical solution by defining system modules, data flow, and communication between frontend and backend.

City Sizzle is designed as a modern client-server based mobile application using Flutter for frontend development and Supabase as the backend service. The system follows a scalable and modular architecture to ensure maintainability, flexibility, and real-time performance.

4.1 System Architecture of City Sizzle

The system architecture of the City Sizzle application defines the overall structure of the system and explains how different components interact with each other to deliver the required functionality. It provides a high-level blueprint of the system, showing how the frontend (mobile application), backend services, and database work together to process user requests, manage data, and ensure real-time communication.

City Sizzle is designed using a **client–server based architecture with a 3-tier structure**, which includes the Presentation Layer, Application Layer, and Data Layer. This architectural style is widely used in modern mobile and web applications because it ensures separation of concerns, better maintainability, scalability, and improved performance.

In this system, the **Flutter mobile application acts as the client (frontend layer)** where users interact with the system. All user actions such as browsing the menu, placing orders, managing cart items, and making payments are performed through this interface. Flutter ensures a responsive and cross-platform user experience, allowing the application to run smoothly on both Android and iOS devices.

The **backend layer is handled by Supabase**, which provides integrated services such as authentication, database management, real-time updates, and API handling. Supabase acts as the core processing unit of the system where all business logic is executed. It validates

user requests, processes orders, manages menu data, and handles secure communication between the frontend and database.

The **database layer is built on PostgreSQL**, which is integrated within Supabase. It is responsible for storing all system data, including user information, food items, cart details, orders, and payment records. The relational structure of the database ensures data consistency, integrity, and efficient retrieval of information.

A key feature of the City Sizzle architecture is its **real-time data synchronization capability**. Any changes made in the database, such as order updates or menu modifications by the admin, are instantly reflected in the user application without requiring manual refresh. This is achieved through Supabase's real-time subscriptions, which continuously listen for database changes and push updates to the frontend.

Another important aspect of the architecture is its **modular design approach**. The system is divided into independent modules such as authentication, menu management, cart system, order processing, payment integration, and admin control panel. Each module performs a specific function and communicates with the backend through API calls. This modularity improves system maintainability and allows future enhancements without affecting the entire system.

Security is also an essential part of the architecture. The system implements **role-based access control**, ensuring that users and administrators have different levels of access. Supabase authentication is used to secure login sessions, while database security rules (Row Level Security) ensure that users can only access their own data. Payment transactions are handled through secure third-party gateways such as JazzCash and EasyPaisa, ensuring safe financial operations.

From a scalability perspective, the architecture is designed to support future expansion. Since Supabase is cloud-based, it allows the system to handle increasing numbers of users and transactions without performance degradation. Additionally, Flutter's lightweight and efficient UI rendering ensures smooth application performance even as the system grows.

In summary, the system architecture of City Sizzle provides a well-structured, secure, and scalable foundation for the application. It ensures smooth interaction between users,

backend services, and the database while maintaining real-time performance and data consistency. This architecture not only supports the current system requirements but also allows room for future enhancements such as AI-based recommendations, live delivery tracking, and advanced analytics.

4.2 Architectural Design

City Sizzle follows a **client–server based 3-tier architecture**, which is widely used in modern mobile applications due to its scalability, maintainability, and separation of responsibilities. This architecture divides the system into distinct layers, where each layer performs a specific role and communicates with the others through defined interfaces.

4.2.1 Architectural Design Overview

The architectural design of City Sizzle is based on a layered approach consisting of three main components:

- Presentation Layer (Flutter Mobile Application)
- Application Layer (Supabase Backend Services)
- Data Layer (PostgreSQL Database)

Each layer is responsible for handling specific tasks and ensures that the system operates in a structured and organized manner. The communication between these layers is handled through API calls and real-time database connections.

4.2.2 Presentation Layer (Frontend – Flutter)

The Presentation Layer represents the user interface of the City Sizzle system, developed using the Flutter framework.

Responsibilities:

- Displaying food menu and categories
- Handling user interactions (login, registration, cart, orders)
- Showing order status and history
- Navigating between screens
- Providing responsive UI for Android and iOS **Key Characteristics:**

- Cross-platform support
- Fast UI rendering
- Widget-based structure
- Real-time updates from backend

This layer ensures that users can interact with the system in an intuitive and user-friendly manner.

4.2.3 Application Layer (Backend – Supabase)

The Application Layer acts as the core processing unit of the system. It is implemented using Supabase [2], which provides backend-as-a-service functionality.

Responsibilities:

- User authentication and session management
- Processing API requests from frontend
- Managing business logic (orders, cart, menu)
- Handling payment status updates
- Real-time communication between client and database

Key Features:

- Built-in REST APIs
- Authentication system (JWT-based)
- Real-time subscriptions
- Secure data handling

This layer ensures that all system operations are processed securely and efficiently.

4.2.4 Data Layer (Database – PostgreSQL)

The Data Layer is responsible for storing and managing all system data using PostgreSQL database integrated with Supabase.

Stored Data Includes:

- User accounts and credentials

- Food menu items and categories
- Cart and order details
- Payment records
- Admin activity logs **Key Characteristics:**

- Relational database structure
- High data consistency and integrity
- Efficient query handling
- Secure data storage

This layer ensures that all information is stored safely and can be retrieved whenever required.

4.2.5 Process Flow Architecture

The data flow in City Sizzle follows a structured sequence from user interaction to backend processing and database storage.

Example: Place Order Flow

- User selects food items in Flutter app
- Items are added to cart
- User confirms order
- Request is sent to Supabase API
- Order is stored in PostgreSQL database
- Admin receives order in dashboard
- Order status is updated
- User receives real-time update
- This flow ensures smooth communication between all system components

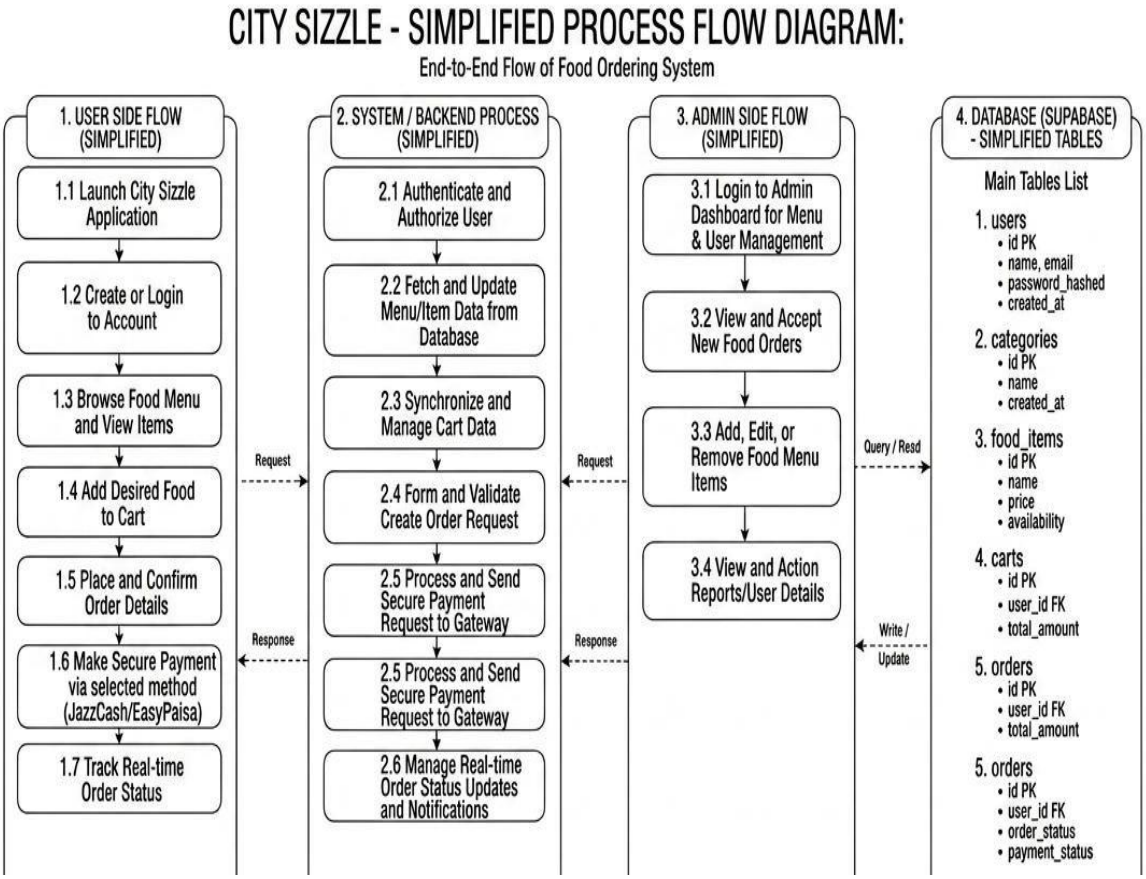

Figure 4.1: Process Flow of City Sizzle

4.2.6 Security Architecture

Security is an important part of City Sizzle architecture to protect user data and transactions.

Security Measures:

- Supabase authentication (JWT tokens)
- Role-based access control (User/Admin separation)
- Password encryption and hashing
- Row Level Security (RLS) in database
- Secure payment gateway integration (JazzCash, EasyPaisa)

These mechanisms ensure that only authorized users can access system resources.

4.2.7 Modular Architecture Design

The system is divided into independent modules to improve maintainability and scalability.

Modules Include:

- Authentication Module
- Menu Management Module
- Cart Management Module
- Order Processing Module
- Payment Module
- Admin Control Module

Each module operates independently and communicates with backend services when required.

4.2.8 Scalability and Performance Design

The architecture is designed to support future growth and increased user load.

Scalability Features:

- Cloud-based backend (Supabase)
- Stateless API communication
- Modular system design
- Real-time database handling **Performance Optimization:**

- Efficient API calls
- Lightweight Flutter UI rendering
- Optimized database queries

This ensures that the system remains fast and responsive even with a growing number of users.

4.3 Design Models

Given the object-oriented programming (OOP) focus of the City Sizzle system, the following design models are used to represent the system structure, behavior, and dynamic interactions. These models help in clearly understanding how different components of the system interact and function together to support core operations such as food ordering, cart management, payment processing, and order tracking.

4.3.1 Class Diagram

A class diagram is a static structure diagram in the Unified Modeling Language [10] (UML) that represents the blueprint of a system by showing its classes, attributes,

methods, and the relationships among them. It is one of the most important diagrams in object-oriented design as it provides a clear and structured view of how the system is organized at the code level.

In the context of the City Sizzle application, the class diagram defines the main entities such as User, Admin, Food Item, Cart, Order, and Payment, along with their attributes and behaviors. It also illustrates the relationships between these entities, such as how a user places orders, how a cart contains food items, and how payments are linked to orders.

Class diagrams are useful for system modeling, database design, and software development as they provide a high-level architectural overview before implementation. They also help developers understand system components and their interactions, making the coding process more organized and efficient.

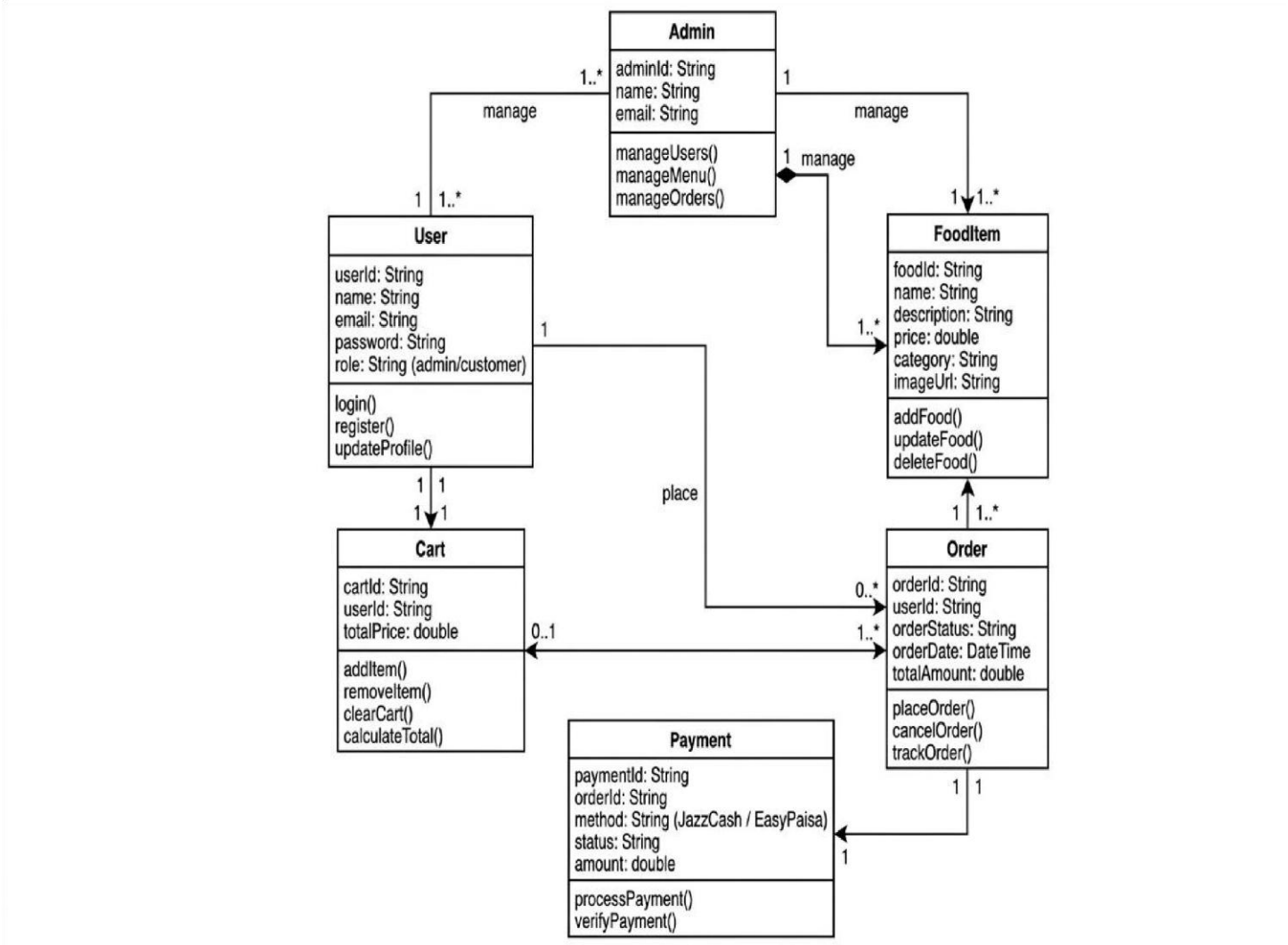

Figure 4.2: Class Diagram

4.3.2 State Transition Diagram

A state transition diagram is a behavioral UML diagram that represents the different states of an object within a system and the transitions between those states based on events or conditions. It uses states (represented as nodes) and transitions (represented as arrows) to model how an object behaves throughout its lifecycle.

In the City Sizzle application, a state transition diagram can be used to model the lifecycle of an **order**, such as: *Order Placed* → *Order Confirmed* → *Preparing* → *Out for Delivery* → *Delivered* → *Completed*. Each state change is triggered by specific events such as payment confirmation, kitchen processing, or delivery updates.

This diagram is important for understanding how the system responds to different conditions over time. It helps in designing reliable workflows and ensures proper tracking of system entities like orders in real-time.

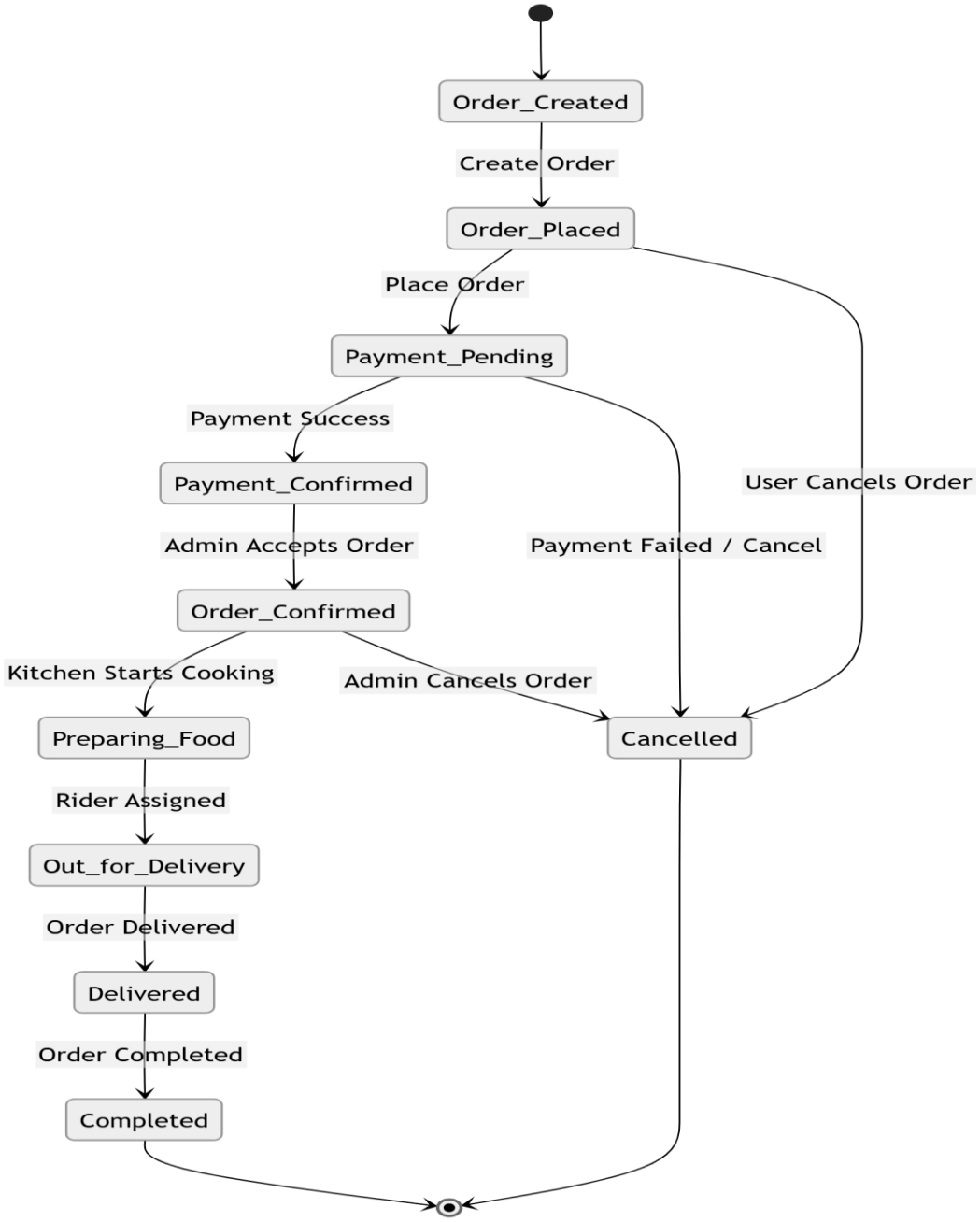

Figure 4.3: State Transition Diagram

Chapter 5 Implementation

5 Implementation

The implementation phase of the City Sizzle project focuses on transforming the system design into a fully functional mobile application. This phase involves the development of the user interface, integration of backend services, and deployment of core functionalities such as order processing, payment handling, and administrative control.

The system is implemented using modern technologies including **Flutter** [1] for frontend development and **Supabase** [2] for backend services. These technologies ensure scalability, performance, and real-time data synchronization.

5.1 Development Environment and Tools

The development environment plays a crucial role in ensuring efficient implementation and testing of the system. The following tools and technologies were used:

Table 5.1: Development Tools

Component	**Technology**
Frontend	Flutter (Dart)
Backend	Supabase
Database	PostgreSQL (via Supabase)
IDE	Android Studio / VS Code
APIs	REST APIs
Payment Integration	JazzCash & EasyPaisa

The use of Flutter enables cross-platform development, allowing the application to run smoothly on Android devices with a single codebase.

5.2 Development Environment and Tools

The development of the City Sizzle application was carried out using a well-structured and modern development environment that supports efficient coding, testing, and deployment. The system was developed using **Flutter**, which provides a cross-platform solution for building high-performance mobile applications with a single codebase. The use of Flutter allowed the developers to design a responsive and user-friendly interface while maintaining consistency across different devices. The application logic and functionality were implemented using **Dart**, which offers strong support for asynchronous programming, making it suitable for real-time operations such as order tracking and data synchronization.

On the backend side, the system utilizes **Supabase**, which serves as a Backend-as-a-Service platform providing authentication, database management, and API handling. Supabase is built on top of **PostgreSQL**, a powerful relational database system that ensures data integrity, reliability, and efficient data retrieval. This backend setup eliminates the need for complex server management and allows seamless communication between the frontend and database through RESTful APIs.

For development and debugging purposes, tools such as **Visual Studio Code** and **Android Studio** were used. Visual Studio Code provided a lightweight and flexible coding environment with various extensions for Flutter development, while Android Studio was primarily used for running emulators, managing SDKs, and testing the application on virtual devices. Version control was handled using **Git**, which enabled efficient collaboration and tracking of code changes throughout the development process.

In addition, API testing and debugging were performed using **Postman**, ensuring that all backend services and endpoints functioned correctly. The integration of digital payment systems such as JazzCash and EasyPaisa was also supported within this environment, enabling secure and reliable financial transactions. Overall, the selected development environment and tools played a crucial role in ensuring the successful implementation of a scalable, efficient, and user-friendly food ordering system

5.3 Role-Based Authentication and Security

Role-based authentication and security are essential components of the City Sizzle application, ensuring that only authorized users can access specific features and perform permitted actions

within the system. The application implements a role-based access control mechanism in which users are assigned predefined roles, such as customer and administrator. Each role is granted access to a specific set of functionalities, which helps in maintaining system integrity and preventing unauthorized operations.

5.3.1 Authentication Mechanism

Authentication in the system is managed through **Supabase**, which provides secure user registration, login, and session management services. When a user signs up or logs into the application, their credentials are verified, and a secure session token is generated. This token is then used to authenticate all subsequent requests, ensuring that only valid users can interact with the system. Passwords are securely stored using encryption techniques, which enhances the protection of user data.

5.3.2 Role-Based Access Control

The system distinguishes between different user roles and assigns permissions accordingly. Customers are allowed to browse menus, place orders, make payments, and track their orders. In contrast, administrators have additional privileges such as managing the food menu, viewing all orders, confirming or rejecting orders, and updating order statuses. Access to administrative features is strictly restricted and can only be granted after proper authentication and role verification, ensuring that sensitive operations remain secure.

5.3.3 Security Measures

Several security measures are implemented to safeguard the system against potential threats. All communication between the application and backend services is conducted over secure channels, ensuring data confidentiality during transmission. Input validation is applied to prevent invalid or malicious data from being processed by the system. Additionally, the integration of digital payment systems such as JazzCash and EasyPaisa is handled securely, ensuring that transactions are verified and protected against fraud or unauthorized access.

5.4 Component-Level Implementation

Component design focuses on breaking down a complex software system into smaller, manageable, and reusable modules. In the City Sizzle application, this modular approach allows different functionalities such as user interaction, order handling, and payment processing to operate independently while remaining interconnected. This structure improves

maintainability, scalability, and system performance by organizing components systematically.

5.4.1 Frontend Components (Flutter)

The user interface of City Sizzle is developed using a component-based architecture in **Flutter**, where each screen and feature is implemented as a reusable widget.

The Home Component is responsible for displaying food categories and menu items dynamically fetched from the backend. The Cart Component manages selected items, quantity updates, and total price calculation in real time. The Order Tracking Component provides users with live updates about their order status, showing stages such as pending, confirmed, preparing, and delivered. The Payment Component allows users to select and process payments using JazzCash or EasyPaisa, ensuring a smooth checkout experience. The Admin Dashboard Component provides administrative control, enabling the admin to manage menu items, monitor orders, and update order statuses efficiently.

5.4.2 Backend Services (Supabase)

The backend of City Sizzle is implemented using Supabase, which provides database management, authentication, and API services.The authentication service handles user registration, login, and session management securely. The database service manages all application data including users, food items, orders, and payments. API endpoints are used to connect the frontend with backend services, allowing real-time data exchange. The real-time feature of Supabase ensures that updates such as order status changes and menu modifications are instantly reflected in the user interface without requiring manual refresh.

5.5 Implementation of Database Schema

The database schema for City Sizzle is implemented using PostgreSQL through Supabase. The schema is designed to maintain data integrity and support efficient data retrieval.

Each entity such as users, orders, food items, and payments is stored in separate relational tables. Relationships between tables are established using foreign keys, ensuring consistency across the system. For example, the Orders table stores references to the User ID and contains related entries in the Order Items table, which stores individual food items associated with each

order. This structure avoids data redundancy and enables efficient querying for operations such as order tracking and reporting.

5.6 Algorithm

An algorithm is defined as a step-by-step procedure used to perform a specific task or solve a problem. In City Sizzle, algorithms are used to manage key system functionalities such as order processing, access control, and payment handling.

5.6.1 Algorithm for Order Processing System

This algorithm controls the complete lifecycle of a customer order from placement to delivery.When a user places an order, the system first collects all selected items from the cart and calculates the total price. The order details are then stored in the database with an initial status of "Pending." Once the order is saved, a notification is sent to the admin dashboard.

The admin reviews the order and updates its status to "Confirmed" or "Rejected." If confirmed, the order progresses through stages such as "Preparing" and "Out for Delivery." Each update is reflected in real time in the user's order tracking screen. The process continues until the order reaches the "Delivered" state, completing the transaction.

5.6.2 Algorithm for Role-Based Access Control (RBAC)

This algorithm ensures secure access to system features based on user roles.During registration, each user is assigned a role such as customer or admin. When a user logs in, the system verifies their credentials and retrieves their role from the database. Based on this role, access permissions are granted.

If the user is a customer, they can browse menus, place orders, and track deliveries. If the user is an admin, additional privileges are enabled, including managing food items, viewing all orders, and updating order statuses. Unauthorized access attempts are restricted by validating roles before executing sensitive operations.

5.6.3 Algorithm for Payment Processing System

This algorithm manages secure transactions using JazzCash and EasyPaisa. When the user proceeds to checkout, the system prompts them to select a payment method. Once selected, the payment request is sent to the respective gateway. The system waits for a response while maintaining a temporary "Processing" state to prevent duplicate transactions.

After receiving the response, the system verifies whether the payment was successful or failed. If successful, the order status is updated to "Confirmed," and the payment record is stored in the database. In case of failure, the system notifies the user and allows them to retry the payment process. This ensures secure and reliable handling of financial transactions.

5.7 User Interface

The User Interface (UI) of the City Sizzle application is designed using **Flutter**, focusing on providing a simple, responsive, and user-friendly experience. The interface is designed to ensure smooth navigation and ease of use for all types of users. The application follows a structured layout with consistent color schemes, typography, and interactive elements to enhance usability.

The UI is developed using a component-based approach, where each screen such as login, menu, cart, and order tracking is implemented as a separate module. This approach improves maintainability and ensures consistency across the application. The interface dynamically updates based on user interactions, such as adding items to the cart or tracking order status in real time.

Screenshots of the application interface are provided in the following sections to illustrate the design and functionality of each component.

5.7.1 Splash and Authentication Interface

The splash screen is the first interface displayed when the application is launched. It provides branding for the City Sizzle application and creates a professional first impression for users. The screen remains visible for a short duration before automatically navigating to the login screen.

The authentication interface serves as the entry point of the City Sizzle application, allowing users to securely access the system. It includes login and registration screens where users can enter their credentials such as email and password.The interface is designed with simplicity and clarity, ensuring that users can easily understand and interact with the system. Proper validation is implemented to handle incorrect inputs, such as invalid email format or wrong password. In such cases, the system displays appropriate error messages to guide the user.

If a user enters incorrect credentials, the system provides immediate feedback through alert messages. Successful authentication redirects the user to the home screen, where they can browse food items and place orders.

Figure 5.2: Splash Screen

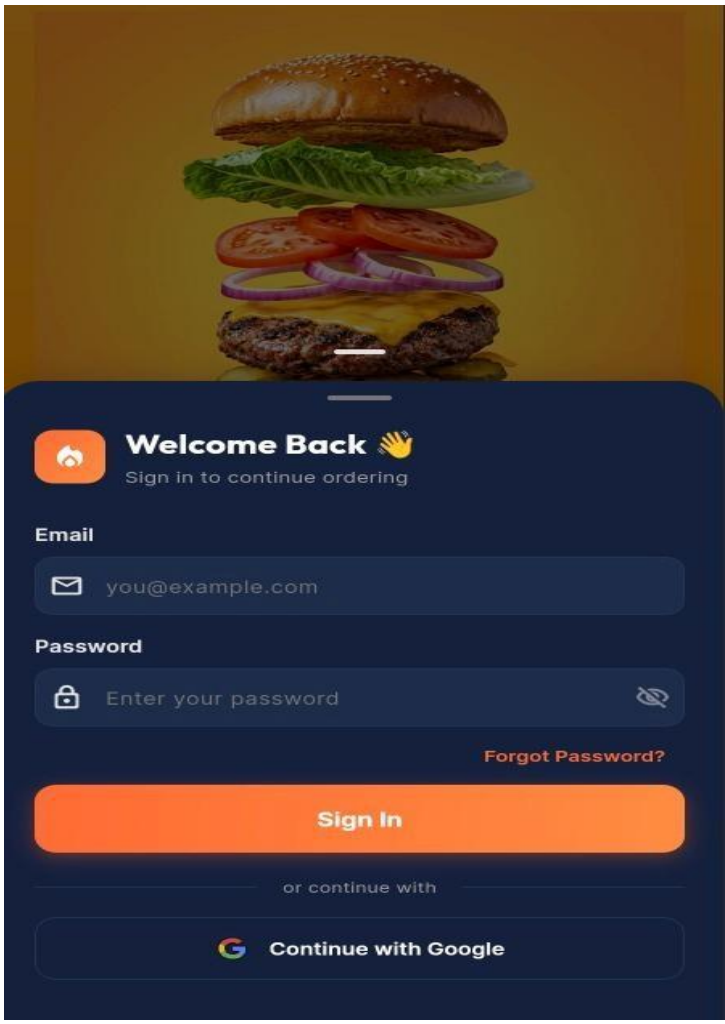

Figure 5.1: Login Screen

5.7.2 Signup Interface

The signup interface enables new users to create an account by providing necessary details such as name, email, and password. The system ensures proper validation to maintain data accuracy and security.

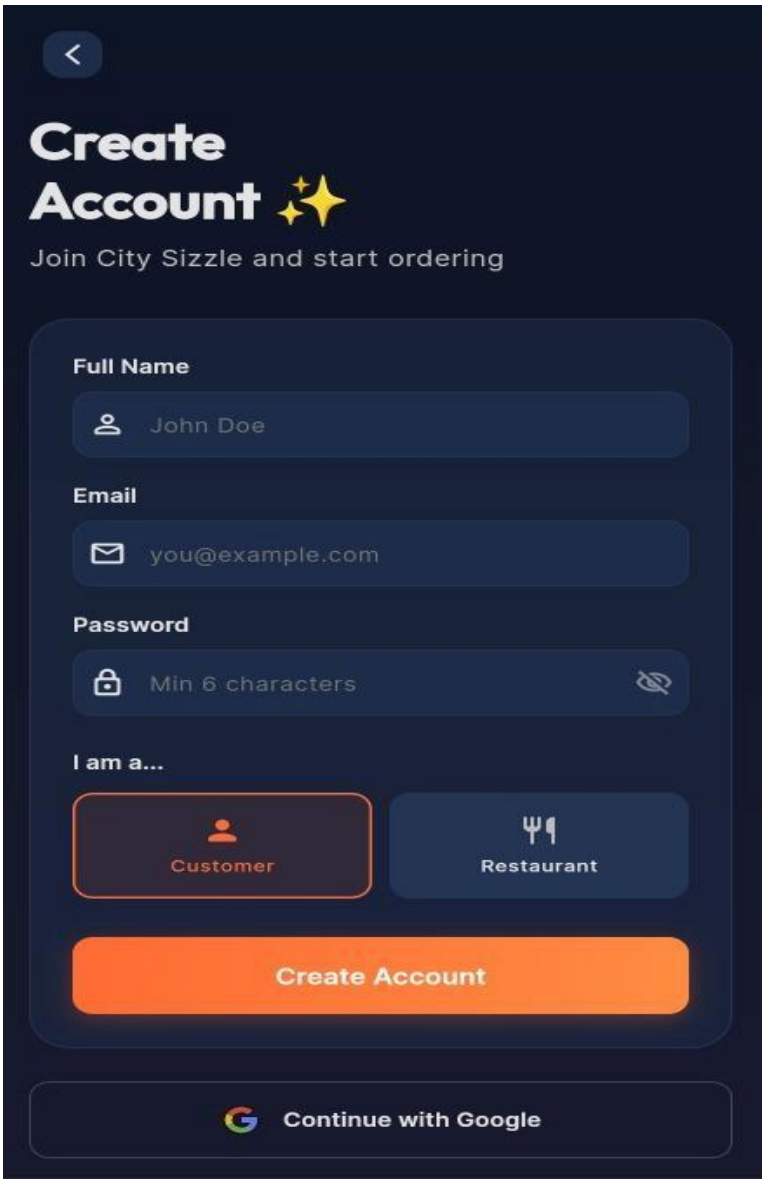

Figure 5.4: Signup Screen

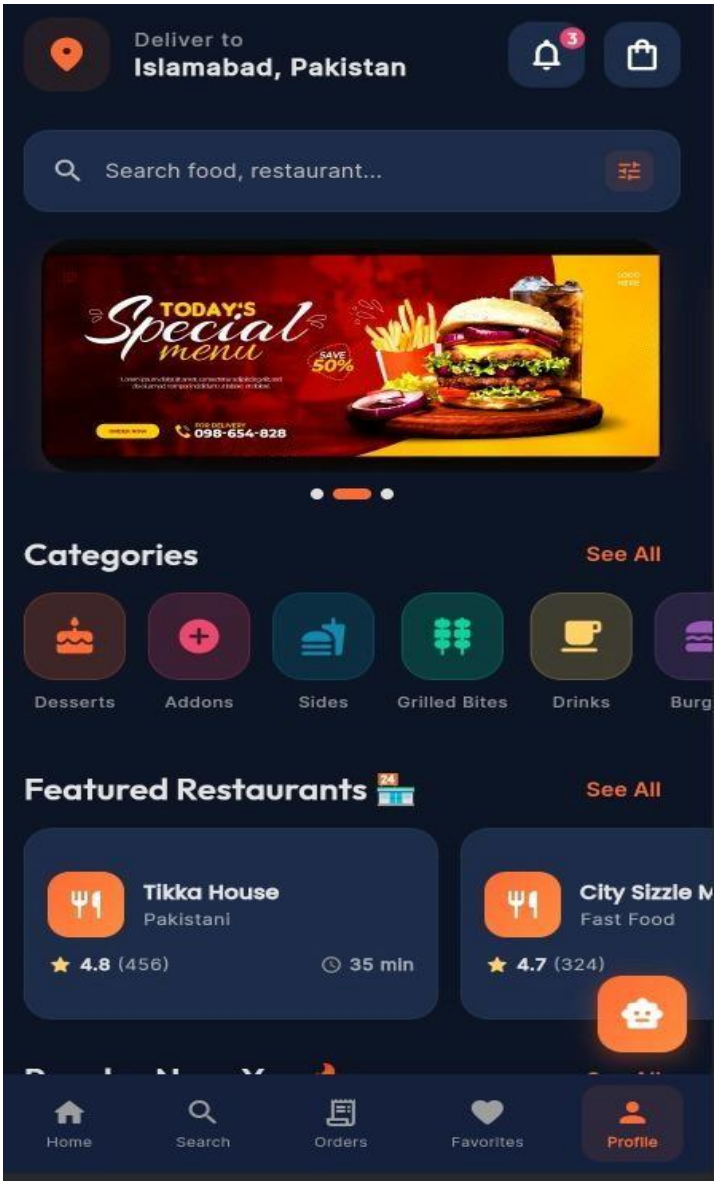

Figure 5.3: Home Screen

5.7.3 Chatbot Interface

The chatbot interface provides interactive support to users by answering queries related to food items, orders, or application usage. It enhances user experience by offering quick assistance without requiring manual support.

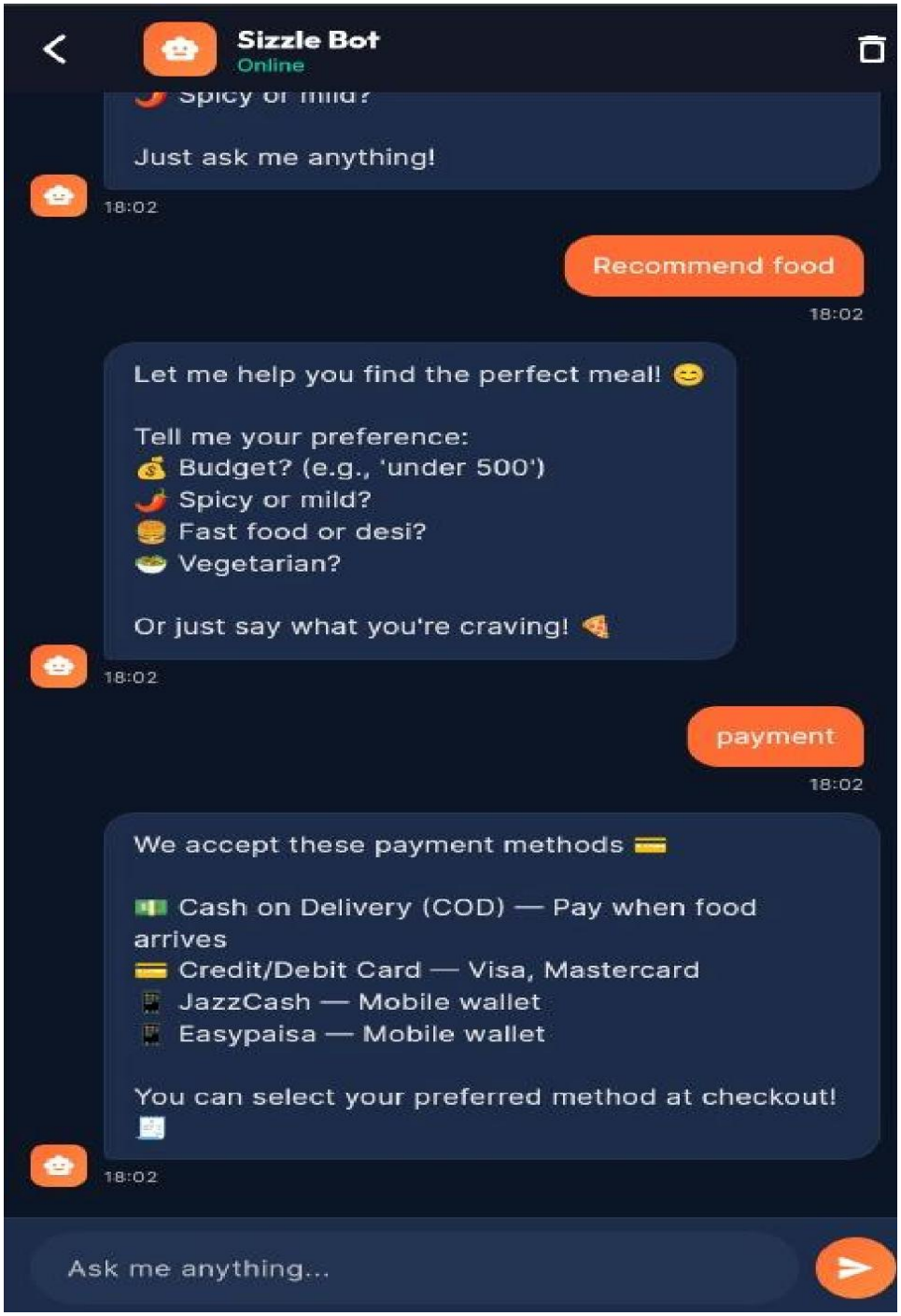

Figure 5.5: Chatbot

5.7.4 Admin Dashboard

The admin dashboard provides administrative control over the system. It allows the admin to monitor system activities, view orders, manage menu items, and control overall operations efficiently.

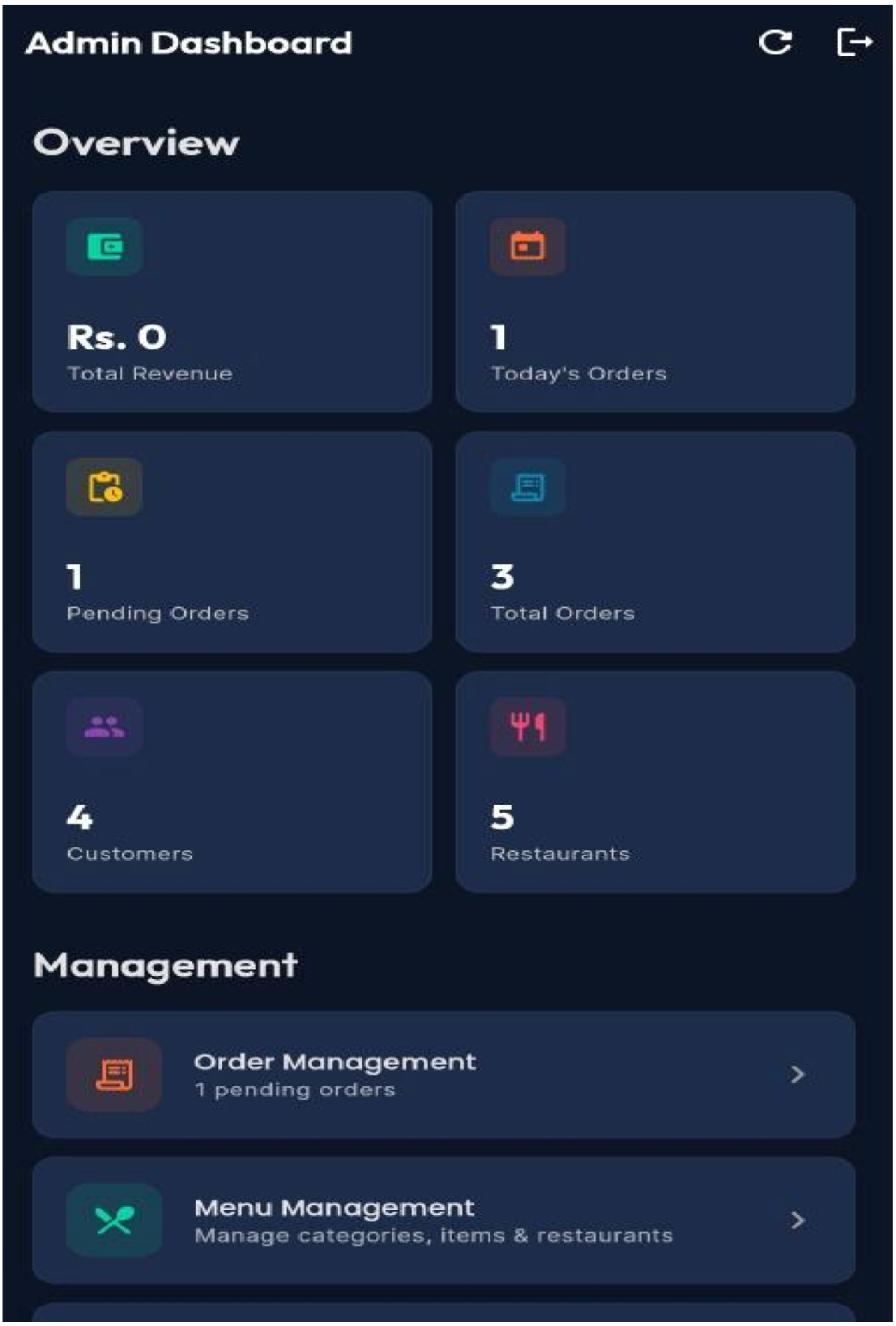

Figure 5.6: Admin Dashboard

Chapter 6: Testing And Evaluation

6 Testing Software

Software testing is the process of evaluating a system to ensure that it meets specified requirements, performs as expected, and is free from defects. In the City Sizzle application, testing was carried out to verify system functionality, usability, performance, and reliability. Manual testing was primarily used to validate the application behavior and user experience across different scenarios. Evaluation refers to the assessment of the system's performance, usability, and overall effectiveness in meeting project objectives.

6.1 Manual Testing

Manual testing was conducted to verify the correctness of system functionalities such as user authentication, food ordering, payment processing, and admin operations. Different test cases were executed to ensure that the application behaves correctly under normal and exceptional conditions.

6.1.1 System Testing

System testing was performed to evaluate the complete working of the City Sizzle application as a whole. The system was tested from user login to order placement and delivery tracking.

This testing ensured that all modules including menu browsing, cart management, payment processing, and admin order management work together correctly. The application successfully handled end-to-end workflows without any major issues.

6.1.2 Unit Testing

Unit testing focuses on testing individual components of the system independently.

6.1.2.1 Unit Testing 1: User Login

Testing Objective: To verify that the system allows only valid users to log in.

Test Case: Enter correct and incorrect login credentials.

Input: Valid email and password / invalid password.

Expected Result: Successful login for valid credentials and error message for invalid login.

Result: Pass.

6.1.2.2 Unit Testing 2: Add to Cart

Testing Objective: To ensure items are correctly added to the cart.

Test Case: User selects a food item and adds it to the cart.

Input: Food item with quantity.

Expected Result: Item appears in cart with correct quantity and price.

Result: Pass.

6.1.3 Functional Testing

Functional testing ensures that system features operate according to requirements.

6.1.3.1 Functional Testing 1: Order Placement

Objective: To verify that users can successfully place orders.

Test Case: Add items to cart and proceed to checkout.

Expected Result: Order is stored in database and confirmation is shown.

Result: Pass.

6.1.3.2 Functional Testing 2: Payment Processing

Objective: To verify JazzCash and EasyPaisa payment functionality.

Test Case: User selects payment method and completes transaction.

Expected Result: Payment is processed and order status updated.

Result: Pass.

6.1.3.3 Functional Testing 3: Admin Order Management

Objective: To verify that admin can manage orders.

Test Case: Admin updates order status.

Expected Result: Status is updated and visible to user.

Result: Pass.

6.1.4 Integration Testing

Integration testing was performed to ensure that different modules of the system interact correctly with each other.

Table 6.1: Integration Testing

Test ID	Integration Path	Input/Trigger	Expected Behavior	Status

01	Flutter ↔ Supabase	Login Request	User authenticated and session created	Pass
02	App ↔ Database	Place Order	Order stored and retrieved correctly	Pass
03	App ↔ Payment Gateway	Payment Request	Payment response received and verified	Pass
04	Admin ↔ Database	Update Order Status	Changes reflected in user interface	Pass

6.2 Automated Testing

Due to the moderate scale of the City Sizzle project, testing was mainly performed manually. However, basic automated testing tools were used for API verification and debugging.

Table 6.2: Automated Testing

Tool Name	Type of Testing	Purpose
Postman	API Testing	Testing backend endpoints
Flutter Testing	UI Testing	Verifying frontend components
Android Studio	Device Testing	Emulator testing

6.3 Evaluation

The evaluation of City Sizzle shows that the system successfully meets its objectives. The application provides a smooth user experience, efficient order processing, and secure payment handling.

The system demonstrates:

- High usability due to simple interface
- Reliable performance during testing
- Accurate data handling through Supabase
- Secure transactions using payment gateways

Overall, the application performs effectively and provides a scalable solution for food ordering systems.

Chapter 7

Conclusion And Future Work

7 Conclusion and Future Work

In this chapter we will discuss about conclusion and future about our Food App.

The primary objective of this project was to design and develop a modern, efficient, and userfriendly food ordering application named City Sizzle. The system aims to simplify the process of ordering food by providing a seamless digital platform that connects customers and administrators while ensuring fast service, secure transactions, and real-time updates.

The application was successfully implemented using **Flutter** for frontend development and **Supabase** for backend services. This combination provided a scalable and high-performance solution capable of handling real-time data and multiple users efficiently. The system incorporates role-based access control, allowing customers to place orders and administrators to manage menu items and process orders securely.

Several key features were successfully achieved during the development of the system. The application provides a smooth and intuitive user interface, enabling users to browse food items, add them to the cart, and complete orders without difficulty. The integration of digital payment systems such as JazzCash and EasyPaisa ensures secure and reliable financial transactions. Additionally, the implementation of an admin panel allows efficient management of orders and menu items, improving operational control. A notable enhancement in the system is the integration of a chatbot, which provides quick assistance to users by answering queries related to food items and application usage. This feature improves user engagement and reduces dependency on manual support.

The system was tested extensively through manual and functional testing, ensuring that all modules perform as expected. The results demonstrate that City Sizzle meets its functional requirements, maintains data integrity, and delivers a reliable user experience. Overall, the project provides a complete and practical solution for modern food ordering systems.

7.1 Future Work

Although the City Sizzle application successfully meets its current objectives, several enhancements can be implemented in future versions to further improve functionality, scalability, and user experience.

7.1.1 Advanced Chatbot and AI Features

The existing chatbot can be further enhanced by integrating advanced artificial intelligence capabilities. Future improvements may include personalized food recommendations based on user preferences, order history, and popular trends. The chatbot can also be upgraded to support voicebased interaction, allowing users to place orders using voice commands, making the application more interactive and user-friendly

7.1.2 Real-Time Order Tracking

Currently, order tracking is based on status updates such as pending, confirmed, and delivered. In future versions, real-time tracking using GPS technology can be implemented, allowing users to monitor the exact location of their delivery in real time. This feature would significantly enhance transparency and user satisfaction.

7.1.3 Cross-Platform Expansion

Although the application is developed for mobile devices, future enhancements may include a web-based version of the system. This would allow users to access City Sizzle from desktops and laptops, increasing accessibility and expanding the user base.

7.1.4 Recommendation and Personalization System

A recommendation system can be introduced to suggest food items based on user preferences, past orders, and popular choices. This feature can improve user engagement and increase sales by providing personalized suggestions.

7.1.5 Loyalty and Reward System

To enhance user retention, a loyalty system can be implemented where users earn points for each order. These points can be redeemed for discounts or special offers, encouraging repeated use of the application.

7.1.6 Enhanced Admin Features

Future improvements may include advanced analytics for the admin panel, such as sales reports, order statistics, and customer insights. This will help administrators make better business decisions and improve service quality.

References

[1] Flutter Development Team, *Flutter Documentation*. Google LLC, 2023. [Online]. Available: https://flutter.dev/docs

[2] Supabase Inc., *Supabase Documentation*. 2023. [Online]. Available: https://supabase.com/docs

[3] State Bank of Pakistan, "Payment systems review," 2023. [Online]. Available: https://www.sbp.org.pk

[4] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.

[5] G. Pigatto, J. G. C. F. Machado, A. S. Negreti, and L. M. Machado, "Have you chosen your request? Analysis of online food delivery companies in Brazil using the PESTEL tool," *British Food Journal*, vol. 119, no. 3, pp. 639– 657, 2017.

[6] V. C. S. Yeo, S. K. Goh, and S. Rezaei, "Consumer experiences, attitude and behavioral intention toward online food delivery (OFD) services," *Journal of Retailing and Consumer Services*, vol. 35, pp. 150–162, 2017.

[7] P. Nawrocki, K. Wrona, M. Marczak, and B. Sniezynski, "A comparison of native and cross-platform frameworks for mobile applications," *Computer*, vol. 54, no. 3, pp. 18–27, 2021.

[8] K. Beck et al., "Manifesto for agile software development," 2001. [Online]. Available: https://agilemanifesto.org

[9] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[10] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. AddisonWesley, 2005.